Circular Dataset Structures

James Foster

2026-10-23

Contents

Details **1**

Motivation **2**

Set up workspace **3**

Divergence from home direction **3**
Fit the model . . . 6
Test the expected angle hypothesis . . . 9

Change in direction **11**
Fit the model . . . 12
Test the direction change hypothesis . . . 18

Change in concentration **20**
Fit the model . . . 22
Test the concentration change hypothesis . . . 25

High inter-individual correlation **27**
Fit the model . . . 30

Low Inter-individual correlation **37**
Fit the model . . . 39
Test shared heading bias hypothesis . . . 44

Variable individual parameters **46**
Fit the model . . . 49

Individual heading changes parameters **53**
Fit the model . . . 58
Test hypothesis of treatment effect . . . 61
Plot model predictions . . . 68

Summary **73**

Details

DESCRIPTION

Examination of different dataset structures that may arise in the study of animal navigation.

INPUTS

`unwrap_functions.R`

OUTPUTS

Plotted results.

REFERENCES

data types (inspired by)

Duelli P & Wehner R (1973) The Spectral Sensitivity of Polarized Light Orientation in Cataglyphis bicolor (Formicidae , Hymenoptera) *Journal of Comparative Physiology* 53(3) 37-53

Papi F & Pardi L (1963). On the Lunar Orientation of Sandhoppers (Amphipoda Talitridae). *Biol. Bull.* 124, 97–105. https://doi.org/10.2307/1539571.

Edrich W, Neumeyer C & von Helversen, O. (1979) "Anti-sun orientation" of bees with regard to a field of ultraviolet light. *J. Comp. Physiol.* 134, 151–157.

Dreyer D, Frost B, Mouritsen H, Günther A, Green K, Whitehouse M, Johnsen S, Heinze S & Warrant E (2018) The Earth's Magnetic Field and Visual Landmarks Steer Migratory Flight Behavior in the Nocturnal Australian Bogong Moth. *Current Biology 28*, 2160-2166.e5. https://doi.org/10.1016/j.cub.2018.05.030.

Baird E, Byrne MJ, Scholtz CH, Warrant E, Dacke M (2010) Bearing selection in ball-rolling dung beetles: is it constant? *Journal of Comparative Physiology A* 196:801–6. https://doi.org/10.1007/s00359-010-0559-8

Giraldo YM, Leitch KJ, Ros IG, Warren TL, Weir PT & Dickinson M H (2018). Sun Navigation Requires Compass Neurons in Drosophila. *Current Biology* 28, 2845-2852.e4.

Kishkinev D, Packmor F, Zechmeister T, Winkler H-C, Chernetsov N, Mouritsen H & Holland RA (2021) Navigation by extrapolation of geomagnetic cues in a migratory songbird. *Current Biology* 12(31) 7, 1563-1569.e4 https://doi.org/10.1016/j.cub.2021.01.051

modelling methods

Sayin S, Couzin-Fuchs E, Petelski I, Günzel Y, Salahshour M, Lee CY, Graving JM, Li L, Deussen O, Sword GA, et al. (2025) The behavioral mechanisms governing collective motion in swarming locusts. *Science* 387(6737):995–791

Gabry J, Češnovar R, Johnson A (2022). cmdstanr: R Interface to 'CmdStan'. https://mc-stan.org/cmdstanr/

Bürkner, P.-C. (2018). Advanced Bayesian Multilevel Modeling with the R Package brms. *The R Journal* 10, 395–411.

Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P. and Riddell, A. (2017). Stan: A Probabilistic Programming Language. *Journal of Statistical Software* 76 doi: 10.18637/jss.v076.i01

Vehtari, A., Gelman, A., and Gabry, J. (2017) Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Statistics and Computing* 27(5), 1413–1432. doi:10.1007/s11222-016-9696-4.

Paananen, T., Piironen, J., Buerkner, P.-C., Vehtari, A. (2021). Implicitly adaptive importance sampling. *Statistics and Computing* 31, 16. doi:10.1007/s11222-020-09982-2.

Motivation

These simulated circular datasets provide examples of effects on mean direction, concentration, and individual differences. Fitted circular models are used to estimate these effects and compare hypotheses about their causes.

Set up workspace

Before generating these examples, we need to load the appropriate functions. Make sure that the `circular`, `cmdstanr` and `brms` are installed, and (on Windows) that you have installed the latest version of Rtools.

```
source('unwrap_functions.R')
```

```
## Loading required package: circular
```

```
##
## Attaching package: 'circular'
```

```
## The following objects are masked from 'package:stats':
##
##     sd, var
```

```
## Loading required package: cmdstanr
```

```
## This is cmdstanr version 0.9.0.9000
```

```
## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr
```

```
## - CmdStan path: C:/Users/James Foster/.cmdstan/cmdstan-2.37.0
```

```
## - CmdStan version: 2.37.0
```

```
##
## A newer version of CmdStan is available. See ?install_cmdstan() to install it.
## To disable this check set option or environment variable cmdstanr_no_ver_check=TRUE.
```

```
## Loading required package: brms
```

```
## Loading required package: Rcpp
```

```
## Loading 'brms' package (version 2.23.0). Useful instructions
## can be found by typing help('brms'). A more detailed introduction
## to the package is available through vignette('brms_overview').
```

```
##
## Attaching package: 'brms'
```

```
## The following object is masked from 'package:stats':
##
##     ar
```

Set up some colours for plotting.

```
col_kappa = '#1E78B5'#colour for kappa parameter
col_rho = '#F08024' #colour for mean vector length
col_sd = '#E74A29' #colour for SD and mean angle
col_sd2 = '#E57461' #colour for other SD heuristics
col_pdf = adjustcolor(col = '#21A885', # colour for probability density
                      alpha.f = 0.7)
col_obs = '#3E1F51' #colour for control observations
col_treat = '#006400' # colour for treatment observations
```

Divergence from home direction

In navigation experiments, return directions are often compared with the expected home direction to determine whether homeward navigation is perturbed. Here, we simulate example data ($n = 20$) generated from a distribution with a mean that differs from the expected direction (0°) by only 15°.

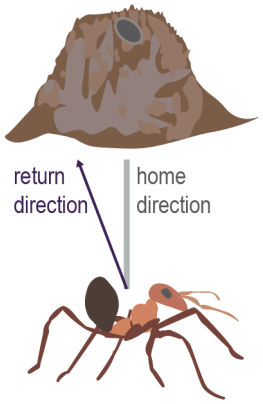


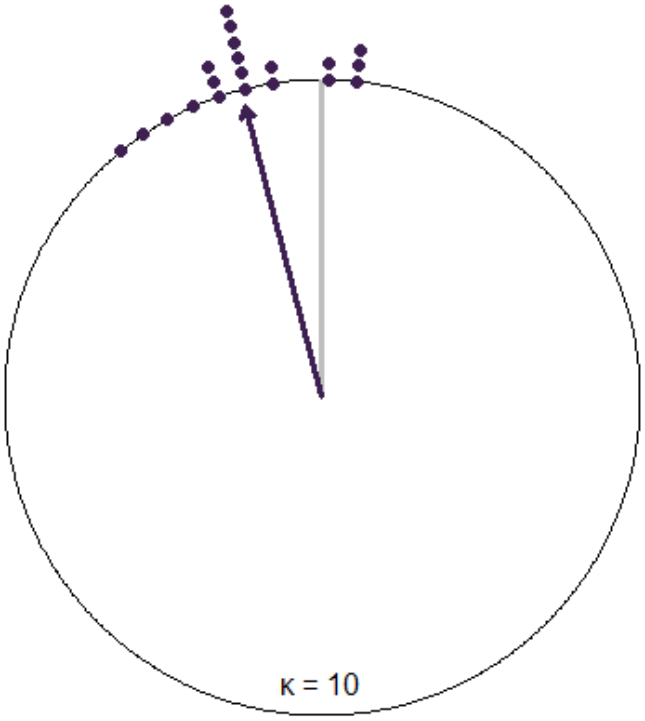
Figure 1: Desert ants of the genus *Cataglyphis* are known for their use of path integration (dead reckoning) to return to their nest. Disruption of visual orientation cues can result in deviations from the expected homewards direction.

```
ndata = 20 #sample size

# circular zero expected angle
c0 = circular(x = 0,
              units = 'degrees',
              rotation = 'clock',
              zero = pi/2)
#true mean angle
c15 = circular(x = -15,
              units = 'degrees',
              rotation = 'clock',
              zero = pi/2)

par(pty = 's') # square axes
par(mar = c(0,0,0,0)) # no margins

cd_divergence = DescriptCplot(m = -15, # population mean
                              k = 10, # high concentration
                              refline = 0, # expected direction
                              ndata = ndata,
                              mvcol = col_obs, #distr. mean same colour as observations
                              sdcol = NA, # don't plot SD
                              denscol = NA, # don't plot prob. density
                              save_sample = TRUE)
```



A common short-cut to assess orientation in an expected direction is to use the v-test modification of the Rayleigh test for uniformity. This tests the hypothesis of uniformity against the alternative of clustering in some direction less than 90° from the expected direction. The data are very far from uniform ($\kappa = 10$), and the mean angle is very close to the expected angle (cosine similarity of `cos(rad(-15)) = 97%`), so the test is significant. Even so, the data show a trend away from the expected angle, 75% of points falling to the left of 0°.

```
mean(cd_divergence < 0)#proportion left of 0°
```

```
## [1] 0.75
```

```
rayleigh.test(cd_divergence, mu = c0)#v-test with 0° expected direction
```

```
##
##        Rayleigh Test of Uniformity
##        Alternative with Specified Mean Direction:  1.570796
##
## Test Statistic:  0.9502
## P-value:  0
```

We can gain more information about any potential differences between the mean direction and the expected angle by fitting a circular generalised linear model (GLM) that estimates the mean direction (μ) for this dataset.

Fit the model

In order to fit a circular model using the circular modulo and softplus links, we need to define a custom family. This family requires two `Stan` functions. One executes the circular modulo transform and the other uses this transform on estimates of μ before calculating the log-probability of data y using the von Mises log-probability-density function (LPDF). This function will be called whenever the family is used, or the prior for a variable is set to `unwrap_von_mises`. Since this applies the circular modulo link function, so we set the link function for μ to "identity", and the link function for κ to "softplus" ($\log(\exp(x)+1)$)), which ensures estimates of κ remain strictly positive.

```
#set up required Stan functions
#circular modulo in Stan code
modulo_circular_fun = stanvar(scode = "
  real modulo_circular(real y) {
    return fmod(y + pi(), 2*pi()) - pi();
  }
",
                              block = 'functions')

#custom likelihood function using the shifted modulo link
stan_unwrap_fun = stanvar(scode = "
  real unwrap_von_mises_lpdf(real y, real mu, real kappa) {
    return von_mises_lpdf(y | modulo_circular(mu), kappa);
  }
  real unwrap_von_mises_rng(real mu, real kappa) {
    return von_mises_rng( modulo_circular(mu) , kappa);
  }
",
                            block = 'functions')
#define the custom family
unwrap_von_mises = custom_family(
  name = "unwrap_von_mises",
  dpars = c("mu",
            "kappa"),
  links = c('identity',#N.B. the link function is defined via the LPD function
            "softplus"),
  lb = c(-pi,#lower bound of mu should be -pi
         0),
  ub = c(pi,#upper bound of mu should be pi
         NA),
  type = "real",#takes continuous response data
)
```

To fit our model, we need to convert our data from degrees to radians. It is also recommended to remove additional formatting by converting from class "circular" to class "numeric". Our formula indicates that each of μ and κ require a single parameter estimate (`~1`). In this data, we expect a mean direction anywhere on the circle (within π of the mean) and high concentration ($\kappa > 1$), so highest prior density is set in that range.

```
#reformat data for BRMS
dt_divergence = data.frame(y = as.numeric(#remove circular formatting
                                  rad(cd_divergence)#convert to radians
                                  )
                           )#make a data frame

#formula for a circular model with no predictors
```

```
form_divergence = bf(y~1,
                     kappa~1,
                     family = unwrap_von_mises)
#unbiased priors
prior_divergence =  prior('normal(0,2*pi())', class = 'Intercept', dpar = 'mu') +
                     prior('normal(3,3)', class = 'Intercept', dpar = 'kappa')

#fit a generic unwrap model
model_divergence = brm(formula = form_divergence,
              data = dt_divergence,
              family = unwrap_von_mises,
              stanvars = stan_unwrap_fun + modulo_circular_fun,
              prior = prior_divergence,
              silent = 2,
              backend = 'cmdstan'#faster and more reliable
  )
```

```
## Loading required namespace: rstan
```

```
summary(model_divergence)
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ 1
##          kappa ~ 1
##    Data: dt_divergence (Number of observations: 20)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept          -0.23      0.08    -0.38    -0.08 1.00     2722     2056
## kappa_Intercept     8.99      2.02     5.34    13.10 1.00     3086     2650
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

Since the mean direction is far from the wrap-point ($-\pi$ or π), the `Rhat` convergence heuristic for the μ `Intercept` can be interpreted on the linear scale. `Rhat` values for both parameters are < 1.01 and effective sample size (ESS) indicates most of the 4000 draws (estimates) explored the posterior distribution efficiently, so we can assume that this model has converged.

To inspect the model's predictions, we can extract the post-warmup draws from the model and plot the posterior distribution of estimates alongside the original data. Since we simulated the data, we can also compare the model estimates with the true parameters (a luxury we do not have for genuine datasets). μ `Intercept` can be plotted as an angle in radians on its original scale. For κ, we need to convert estimates to the correct scale using the `softplus` transform, and then calculate its mean-vector-length equivalent using the `A1` function. There are `4000` posterior estimates, so rather than plotting all of them we'll summarise them as the medians of μ and κ estimates and the normalised 2D kernel density (red and yellow shaded region) of their mean-vector equivalent.

```
#extract all "draws" of the posterior as a data.frame
draws_divergence = as_draws_df(model_divergence)

#open the plot with two panels
```

```r
par(pty = 's')
par(mar = c(0,0,0,0),
    mfrow = c(1,2))
#plot the raw data
PCfun(cd_divergence,
      col = col_obs,
      sep = 0.05,
      shrink = 1.25,
      plot_rho = FALSE)
#plot the estimate density
Draws2Cont(draws_divergence,
           x_string = )
#add the mean vector of the distribution used to simulate the data
arrows.circular(x = circular(-15,
                             units = 'degrees',
                             rotation = 'clock',
                             zero = pi/2),
                y = A1(10),
                col = col_obs,
                lwd = 5,
                length = 0.1/1.25
)

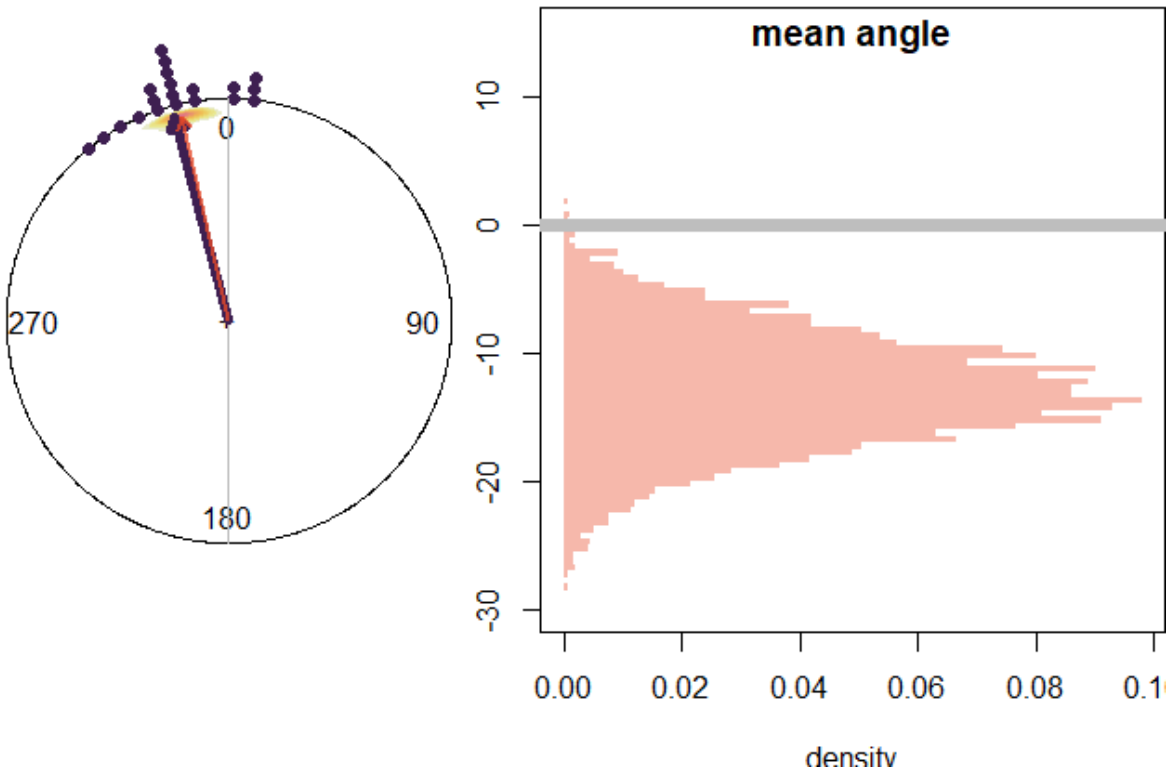
#add the median mean-vector estimate
with(draws_divergence,
     arrows.circular(x = median.circular(x =
                                           circular(Intercept,
                                 rotation = 'clock',
                                 zero = pi/2)
                                 ),
                     y = A1(softplus(x = median(x = Intercept_kappa))),
                     lwd = 2,
                     length = 0.1/1.25,
                     col = adjustcolor(col_sd, alpha.f = 200/255))
)

#calculate the misaligned posterior density
with(draws_divergence,
     {
       paste0('mu left of expected angle: ',
              mean(Intercept < 0)*100, '%')
      }
     ) #nearly all estimates suggest a leftwards turn
```

```
## [1] "mu left of expected angle: 99.9%"
```

```r
#add a histogram of estimates for mu-intercept
with(draws_divergence,
     {
      VertHist(data = deg(Intercept),
               main = '\nmean angle',
               ylim = c(-30, 15),
               col = adjustcolor(col_sd, alpha.f = 100/255),
               cex.axis = 0.7)
```

```
    }
)
#line showing expected direction
abline(h = 0,
       col = 'gray',
       lwd = 7)
```



The fitted model indicates that the mean direction is to the left of the expected direction in $> 99\%$ of μ estimates.

Test the expected angle hypothesis

To test the hypothesis that the true mean direction falls in the expected direction, we can fit another model where this is explicitly stated. To inform BRMS that we do not want to estimate the mean direction, we use the notation `y~0`, which removes the intercept parameter. We can now estimate κ, *assuming* that the mean direction is 0°, using the same prior for κ as before.

```
#fit a model with mean at 0
model_expect = brm(formula = bf(y~0,
                                kappa~1),
                   data = dt_divergence,
                   family = unwrap_von_mises,
                   prior = prior_divergence[2,], # just the kappa prior,
                   stanvars = stan_unwrap_fun + modulo_circular_fun,
                   silent = 2,
                   backend = 'cmdstan'
```

```
                )

#add Leave-One-Out Cross-Validation
loo_divergence = loo(model_divergence)
loo_expected = loo(model_expect)
loo_compare(loo_divergence, loo_expected)
```

```
##                  elpd_diff se_diff
## model_divergence  0.0       0.0
## model_expect     -4.2       1.7
```

```
#ELPD difference between two models (first minus second) and its standard error,
#calculated from pointwise ELPD in the same way as loo_compare
ElpdDiff = function(loo1, loo2, digits = 1)
{
  d = loo1$pointwise[, 'elpd_loo'] - loo2$pointwise[, 'elpd_loo']
  paste(formatC(sum(d), format = 'f', digits = digits), '±',
        formatC(sqrt(length(d)) * sd(d), format = 'f', digits = digits))
}
```

The model for a divergent mean direction has higher expected log predictive density (ELPD) than the model for the expected direction. This ELPD difference is $> 2\times$ larger than its standard error, so we can state with some confidence that the hypothesis of a divergence from the expected direction is more likely than the hypothesis of alignment with the expected direction.

In this case, the expected angle model is straightforward to fit because the expected angle is 0°, which is also the default intercept for a `y~0` (no-intercept) model. If the expected direction were not 0°, we would need to adjust the data to align the expected angle with the `0` intercept, then fit both models (expected direction and divergent direction) to the adjusted data. Here is an example with the expected angle at -15°.

```
#align the data relative to the expected angle
dt_aligned = within(dt_divergence,
                    {
                    y = y - rad(-15)  #subtract the expected angle
                    }
                    )


#we now place our expected angle at -15°, the true mean
model_true = brm(formula = bf(y ~ 0,
                              kappa~1),
                 data = dt_aligned,
                 family = unwrap_von_mises,
                 prior = prior_divergence[2,], # just the kappa prior
                 stanvars = stan_unwrap_fun + modulo_circular_fun,
                 silent = 2,
                 backend = 'cmdstan'
)

#this model should be compared with another fitted to the
#_same data_
model_false =  brm(formula = bf(y ~ 1,
                              kappa~1),
                 data = dt_aligned,
                 family = unwrap_von_mises,
```

```
                prior = prior_divergence, # the kappa and mu priors
                stanvars = stan_unwrap_fun + modulo_circular_fun,
                silent = 2,
                backend = 'cmdstan'
)

#both models converge well with rhats [1.000, 1.002]
sm_truefalse = lapply(list(true = model_true, false = model_false), summary)
#their mu estimates are very close, differing by only 2°, with overlapping CI.
round( deg(
  sm_truefalse$false$fixed['Intercept',
                                    1:4]
  ) )
```

```
##           Estimate Est.Error l-95% CI u-95% CI
## Intercept        2         4       -6       11
```

```
#add Leave-One-Out Cross-Validation
loo_true = loo(model_true)
loo_false = loo(model_false)
loo_compare(loo_true, loo_false)
```

```
##             elpd_diff se_diff
## model_true   0.0       0.0
## model_false -0.8       0.4
```

Now the expected direction model has higher predictive power, even though both find a similar mean direction. If the divergence model had higher predictive power, but within one standard error of the expected direction model, we could not rule out the possibility that the expected direction is the true direction, though evidence would be weaker.

Change in direction

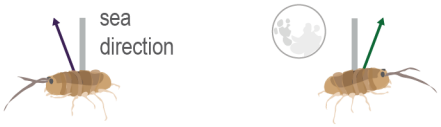


Figure 2: Sandhoppers (*Talitridae*) forage along beaches, following the movement of the sea to avoid drying out. Their movement is guided by celestial cues, including the moon.

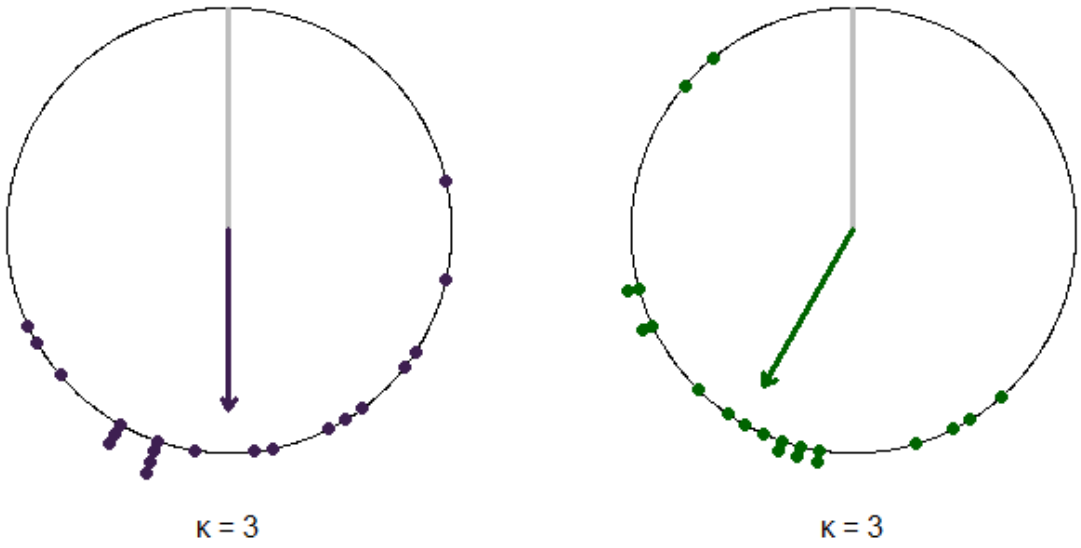
If the expected direction is not known, it can be estimated from control data. This estimate can then be compared with treatment data, to determine whether the treatment alters orientation. In this example, the treatment induces a rightward turn of 30°, changing μ from 180° to 210°. Concentration is intermediate ($\kappa = 3$). The two simulated distributions have different random number seeds, to avoid producing two identical distributions of angles with a fixed shift.

```
par(pty = 's')
par(mar = c(0,0,0,0))
par(mfrow = c(1,2))

cd_control = DescriptCplot(m = 180,
                    k = 3,
                    ndata = ndata,
                    mvcol = col_obs,
```

```
                        sdcol = NA,
                        denscol = NA,
                        refline = c0,
                        save_sample = TRUE,
                        titleline = -1,#needs to be lower when data are South
                        seed = 1539571)#DOI Papi & Pardi

cd_treatment = DescriptCplot(m = 210,
                          k = 3,
                          ndata = ndata,
                          pcol = col_treat,
                          mvcol = col_treat,
                          sdcol = NA,
                          denscol = NA,
                          refline = c0,
                          save_sample = TRUE,
                          titleline = -1,#needs to be lower when data are South
                        seed = 0120810506) #ISBN Batschelet 1981
```



Fit the model

To model the change in direction, we need to account for potential differences in both direction and concentration between the control and treatment. We can combine our two sets of angles together into one vector y, that is aligned with the vector x, which specifies in Boolean logic (yes: 1, no: 0) whether the treatment was applied . *N.B.* For stability, a slightly narrower prior on `b_Intercept`, $\mathcal{N}(0, 3\pi/2)$, is used. This reduces

the region of 50% prior density from [-240°, 240°] to [-180°, 180°].

```
dt_delta = data.frame(y = rad(
                            as.numeric(
                            c(cd_control,
                              cd_treatment)
                            ) ),
                      x = c(rep(0, length(cd_control)),
                            rep(1, length(cd_treatment)))
                      )

#formula for a circular model with predictor x
#N.B. "0 + Intercept" prevents BRMS from centring predictors,
#so that the intercept is the control, not the average across conditions
form_delta = bf(y ~ 0 + Intercept + x,
                kappa ~ 0 + Intercept + x,
                family = unwrap_von_mises)

prior_delta = prior('normal(0,3*pi()/2)', class = 'b', coef = 'Intercept', dpar = 'mu') +
        prior('normal(0,pi()/2)', class = 'b', coef = 'x', dpar = 'mu') +
        prior('normal(3,3)', class = 'b', coef = 'Intercept', dpar = 'kappa') +
        prior('normal(0,3)', class = 'b', coef = 'x', dpar = 'kappa')


model_delta =  brm(formula = form_delta,
               data = dt_delta,
               family = unwrap_von_mises,
               prior = prior_delta, # the kappa and mu priors
               stanvars = stan_unwrap_fun + modulo_circular_fun,
               silent = 2,
               backend = 'cmdstan'
)
summary(model_delta)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

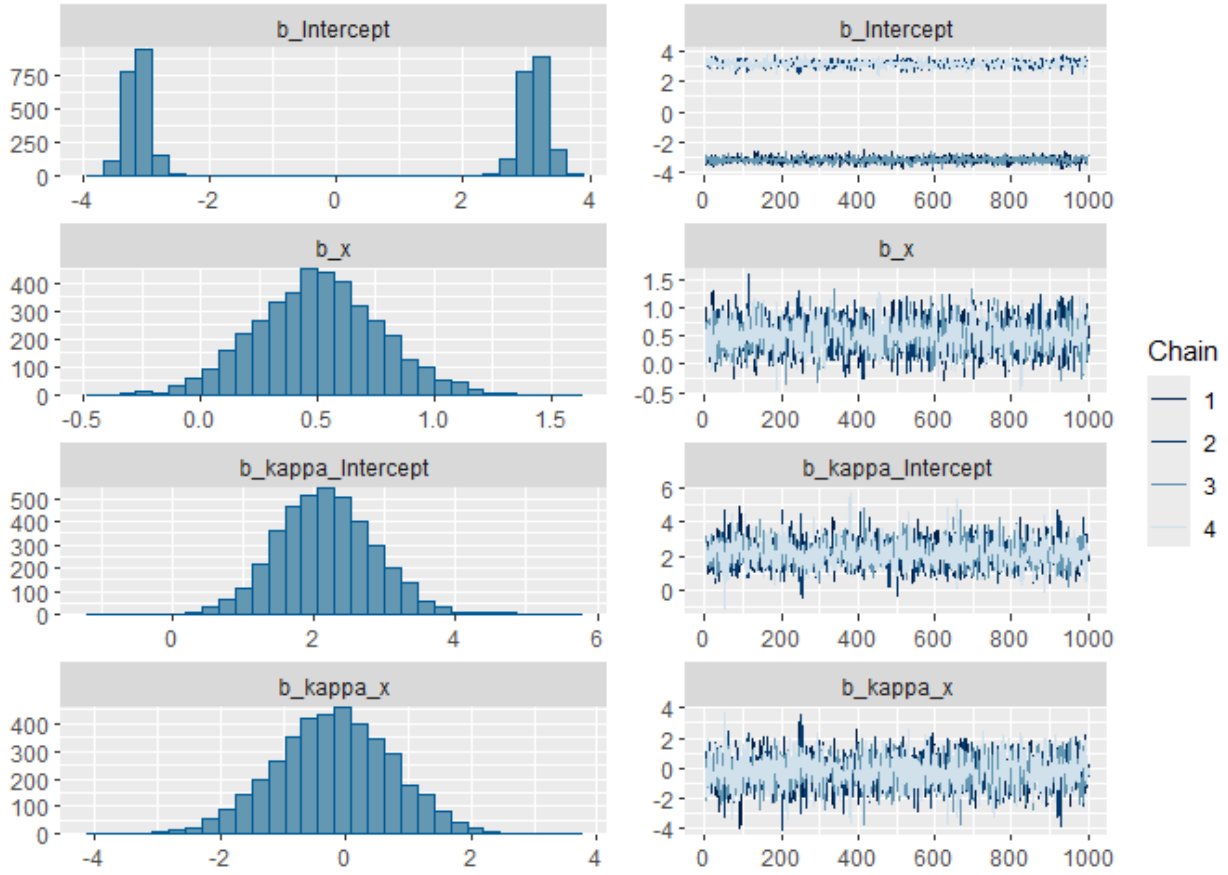
```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ 0 + Intercept + x
##          kappa ~ 0 + Intercept + x
##    Data: dt_delta (Number of observations: 40)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept          -0.01      3.14    -3.43     3.44 1.73        6      158
## x                   0.50      0.27    -0.02     1.03 1.00     2000     2092
## kappa_Intercept     2.17      0.70     0.85     3.59 1.00     2108     2299
## kappa_x            -0.20      0.96    -2.13     1.63 1.00     2363     2525
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
```

```
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

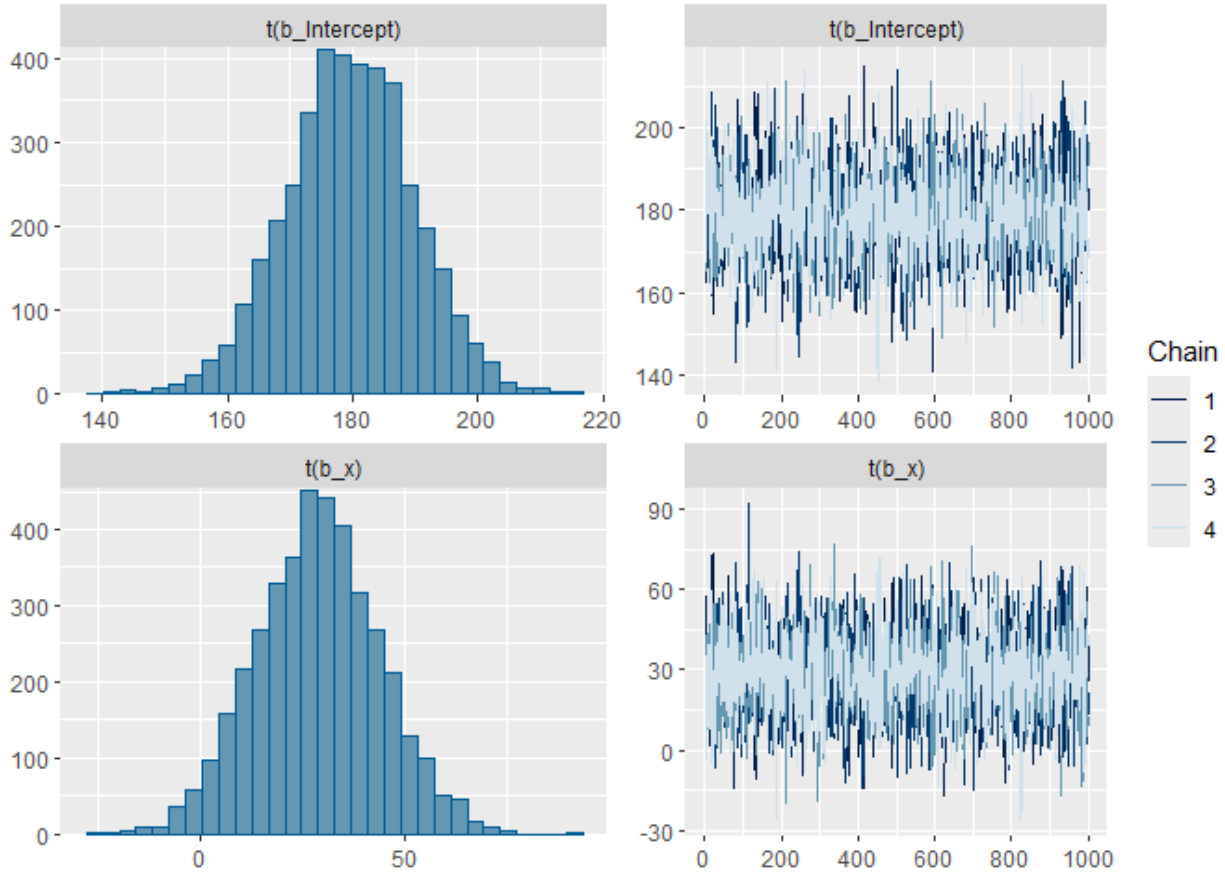
N.B. `brms` warns that parts of the model have not converged (some `Rhat` > 1.05) whenever estimates of a circular parameter have wrapped around the circle in some chains but not others. This is expected for these models, and convergence of circular parameters should instead be checked after unwrapping (`UnwrapRhats`), as below.

Because the mean direction for the control now falls precisely at the boundary of $(-\pi, \pi)$, estimates for the `b_Intercept` parameter with equal likelihood could appear in both positive and negative directions. The same is also possible for the coefficient `b_x` (treatment change in angle). If we unwrap these estimates, we can confirm that they converged towards the same angle, even if they moved in different directions.

```
plot(model_delta)
```




```
#unwrap estimates and convert to degrees
plot(x = model_delta,
     variable = c('b_Intercept','b_x'),
     transform = unwrap_circular_deg)
```




```
#print the rhat for unwrapped estimates
UnwrapRhats(model_delta,
            variable = c('b_Intercept','b_x'),
            regex = FALSE
            )
```

```
## b_Intercept         b_x
##     1.00123     1.00002
```

Now that we have established that the model has converged, we can inspect the model predictions. To do this we need to apply the model formula to the estimates. That means that in the control condition we use estimates of `b_Intercept` (μ) and `b_kappa_Intercept`, but for the treatment we need `b_Intercept` + `b_x` (effect of treatment on μ) and `b_kappa_Intercept` + `b_kappa_x` (effect of treatment on κ).

```
draws_delta = as_draws_df(model_delta)

par(pty = 's')
par(mar = c(0,0,0,0),
    mfrow = c(1,3))
PCfun(cd_control,
      col = col_obs,
      sep = 0.05,
      shrink = 1.25,
      plot_rho = FALSE)
Draws2Cont(draws_delta,
           x_string = 'sin(b_Intercept)*A1(softplus(b_kappa_Intercept))',
           y_string = 'cos(b_Intercept)*A1(softplus(b_kappa_Intercept))',
```

```
)
arrows.circular(x = circular(180,
                             units = 'degrees',
                             rotation = 'clock',
                             zero = pi/2),
                y = A1(3),
                col = col_obs,
                lwd = 5,
                length = 0.1/1.25
)
with(draws_delta,
     arrows.circular(x = median.circular(x =
                             circular(b_Intercept,
                                  rotation = 'clock',
                                  zero = pi/2)
                                  ),
                     y = A1(softplus(x = median(x = b_kappa_Intercept))),
                     lwd = 2,
                     length = 0.1/1.25,
                     col = adjustcolor(col_sd, alpha.f = 200/255))
)

PCfun(cd_treatment,
      col = col_treat,
      sep = 0.05,
      shrink = 1.25,
      plot_rho = FALSE)
Draws2Cont(draws_delta,
           x_string = 'sin(b_Intercept+b_x)*A1(softplus(b_kappa_Intercept+b_kappa_x))',
           y_string = 'cos(b_Intercept+b_x)*A1(softplus(b_kappa_Intercept+b_kappa_x))',
)
arrows.circular(x = circular(205,
                             units = 'degrees',
                             rotation = 'clock',
                             zero = pi/2),
                y = A1(3),
                col = col_treat,
                lwd = 5,
                length = 0.1/1.25
)
with(draws_delta,
     arrows.circular(x = median.circular(x =
                             circular(b_Intercept + b_x,
                                  rotation = 'clock',
                                  zero = pi/2)
                                  ),
                     y = A1(softplus(x = median(x = b_kappa_Intercept +
                                                  b_kappa_x))),
                     lwd = 2,
                     length = 0.1/1.25,
                     col = adjustcolor(col_sd, alpha.f = 200/255))
)
#proportion of estimates suggesting a rightwards turn
```

```r
prop_right_delta = with(draws_delta, mean(mod_circular(b_x) > 0))
paste0(prop_right_delta*100, '%')
```

```
## [1] "97.175%"
```

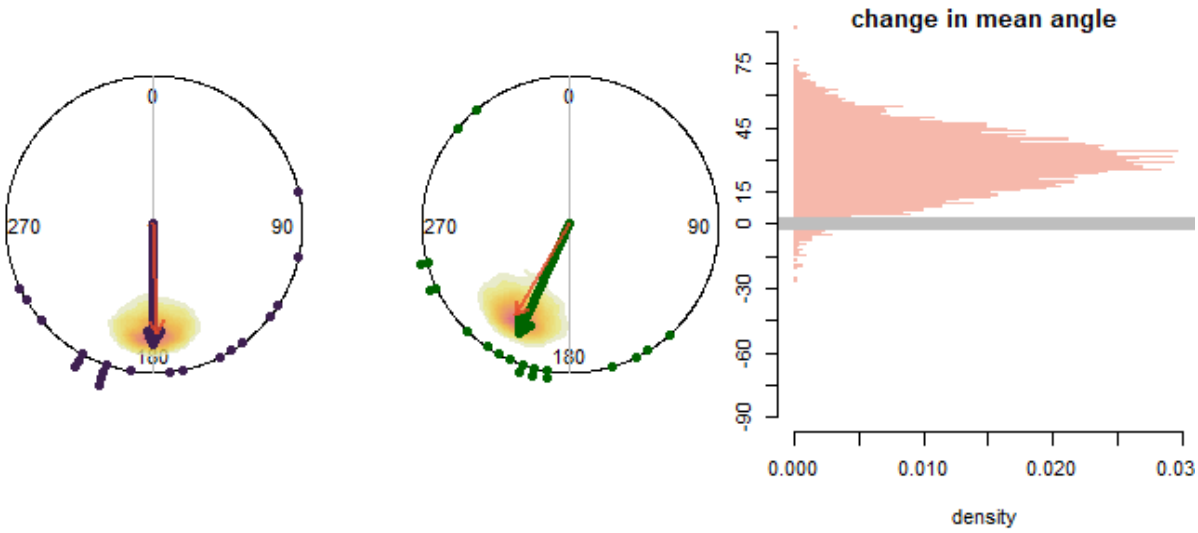
```r
median_turn_delta = with(draws_delta,
                         deg(median.circular(x =
                             circular(b_x,
                                  template = 'none')
                                  )
                           ) )
paste0(round(median_turn_delta), '°') #the size recovered is similar to the simulated turn of 210-180
```

```
## [1] "29°"
```

```r
#95% CI of the turn (unwrapped, degrees)
ci_turn_delta = with(draws_delta,
                     quantile(unwrap_circular_deg(b_x), probs = c(0.025, 0.975)) )
round(ci_turn_delta)
```

```
##  2.5% 97.5%
##    -1    59
```

```r
with(draws_delta,
     VertHist(data = Mod360.180(deg(b_x)),
              main = 'change in mean angle',
              ylim = c(-90, 90),
              col = adjustcolor(col_sd, alpha.f = 100/255),
              axes = F,
              cex.axis = 0.7))
abline(h = 0,
       col = 'gray',
       lwd = 7)
axis(1)
axis(2, at = -6:6*15)
```



Uncertainty in the example data results in a wide range of posterior estimates for the treatment direction. Nonetheless, 97.2% of estimates for change in μ are to the right of 0°, suggesting a rightwards change in direction.

Test the direction change hypothesis

We can investigate this further using model comparison, by fitting models with only treatment effects on κ, and no effects of treatment on μ or κ. We do this by setting the formula for μ and κ to `~ 0 + Intercept` (estimate intercept only).

```
#model with effects of treatment on kappa, but not mu
model_kappadelta =  brm(formula = bf(y ~ 0 + Intercept,
                                     kappa ~ 0 + Intercept + x),
                data = dt_delta,
                family = unwrap_von_mises,
                prior = prior_delta[c(1,3:4),], # the all kappa & 1st mu prior
                stanvars = stan_unwrap_fun + modulo_circular_fun,
                silent = 2,
                backend = 'cmdstan'
)
#model with no effects of treatment
model_nodelta =  brm(formula = bf(y ~ 0 + Intercept,
                                  kappa ~ 0 + Intercept),
                data = dt_delta,
                family = unwrap_von_mises,
                prior = prior_delta[c(1,3),], # intercept kappa & mu priors
```

```
              stanvars = stan_unwrap_fun + modulo_circular_fun,
              silent = 2,
              backend = 'cmdstan'
)
#all models converge well
print(
  list(`treat. kappa` =
    summary(model_kappadelta),
    `unwrapped rhat` =
    UnwrapRhats(model_kappadelta,
                variable = c('b_Intercept'),
                regex = FALSE
                ),
    `no treat.` =
    summary(model_nodelta),
    `unwrapped rhat no treat.` =
    UnwrapRhats(model_nodelta,
                variable = c('b_Intercept'),
                regex = FALSE
                )
    ),
  digits = 3
)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
## $`treat. kappa`
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ 0 + Intercept
##          kappa ~ 0 + Intercept + x
##    Data: dt_delta (Number of observations: 40)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS Tail_ESS
## Intercept         -1.338     2.719   -3.168    3.551 1.529        7       31
## kappa_Intercept    2.036     0.683    0.749    3.449 1.003     2116     2218
## kappa_x           -0.227     0.964   -2.141    1.704 1.004     2030     2233
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
##
## $`unwrapped rhat`
## b_Intercept
##           1
##
```

```
## $`no treat.`
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ 0 + Intercept
##          kappa ~ 0 + Intercept
##    Data: dt_delta (Number of observations: 40)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS Tail_ESS
## Intercept         -1.330     2.720   -3.151    3.553 1.531        7       30
## kappa_Intercept    1.900     0.468    0.998    2.843 1.001     2967     2391
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
##
## $`unwrapped rhat no treat.`
## b_Intercept
##           1
```

```
loo_delta = loo(model_delta)
loo_kappadelta = loo(model_kappadelta)
loo_nodelta = loo(model_nodelta)
lc_delta = loo_compare(loo_delta, loo_kappadelta, loo_nodelta)
print(lc_delta)
```

```
##                  elpd_diff se_diff
## model_nodelta     0.0       0.0
## model_delta      -0.1       1.8
## model_kappadelta -1.0       0.3
```

In this case there is uncertainty about the effect of treatment on direction. Although most estimates from the model with a change in direction indicate a rightwards turn (97.2% of estimates; 95% CI: -1° to 59°), these differences could also be explained well by a null model with no change (ELPD difference: -0.1 ± 1.8). Since the median estimate (29°) is close to the true value of 30°, the model's estimates appear to be robust, if not conclusive. Conversely, there is no evidence for an effect of treatment on concentration, since `model_kappadelta` has lower predictive density than the null model (`model_nodelta`; ELPD difference: -1.0 ± 0.3), although this difference is small.

Change in concentration

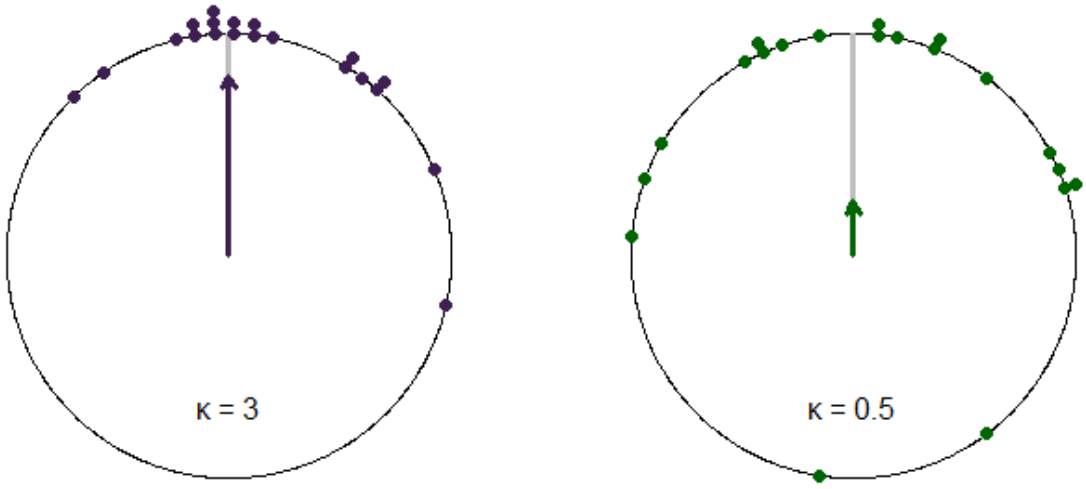


Figure 3: Honeybees performing a "waggle dance" on a horizontal observation hive that view a green light-spot, orient their dances as if viewing the sun. The concentration of a series of dance directions decreases when the brightness of this light spot is reduced.

An experimental treatment may also affect concentration independently of mean direction. In this example the simulated data for the treatment has exactly the same μ (0°) as the control, but κ decreases by $> 80\%$.

```r
par(pty = 's')
par(mar = c(0,0,0,0))
par(mfrow = c(1,2))

cd_3 = DescriptCplot(m = 0,
                     k = 3,
                     ndata = ndata,
                     mvcol = col_obs,
                     sdcol = NA,
                     denscol = NA,
                     refline = c0,
                     save_sample = TRUE)
cd_0.5 = DescriptCplot(m = 0,
                       k = 0.5,
                       ndata = 20,
                       pcol = col_treat,
                       mvcol = col_treat,
                       sdcol = NA,
                       denscol = NA,
                       refline = c0,
                       save_sample = TRUE)
```



Fit the model

By default, the assumptions for this model would be the same as for the change in angle: we want to be able to account for any potential effects of the treatment on both μ and κ, even if we suspect that the treatment only affects one parameter. We can therefore use the same model as for the change in angle.

```
dt_conc = data.frame(y = rad( as.numeric( c(cd_3, cd_0.5) ) ),
                     x = c(rep(0, length(cd_3)), rep(1, length(cd_0.5))) )

#formula for a circular model with predictor x
form_conc = bf(y ~ 0 + Intercept + x,
               kappa ~ 0 + Intercept + x,
               family = unwrap_von_mises)

prior_conc = prior('normal(0,3*pi()/2)', class = 'b', coef = 'Intercept', dpar = 'mu') +
               prior('normal(0,pi()/2)', class = 'b', coef = 'x', dpar = 'mu') +
               prior('normal(3,3)', class = 'b', coef = 'Intercept', dpar = 'kappa') +
               prior('normal(0,3)', class = 'b', coef = 'x', dpar = 'kappa')

model_conc = brm(formula = form_conc,
                 data = dt_conc,
                 family = unwrap_von_mises,
                 prior = prior_conc, # the kappa and mu priors
                 stanvars = stan_unwrap_fun + modulo_circular_fun,
                 silent = 2, backend = 'cmdstan' )
summary(model_conc)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ 0 + Intercept + x
##          kappa ~ 0 + Intercept + x
##    Data: dt_conc (Number of observations: 40)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## Intercept           1.76      2.72    -0.06     6.65 1.53        7       32
## x                  -0.07      0.33    -0.73     0.54 1.00     2275     1888
## kappa_Intercept     3.36      0.93     1.68     5.32 1.00     1601     1981
## kappa_x            -2.44      1.12    -4.83    -0.36 1.00     1626     1771
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
#proportion of estimates for change in mu to the left of 0°
prop_left_conc = mean(mod_circular(as_draws_df(model_conc)$b_x) < 0)
paste0(prop_left_conc*100, '%')
```

```
## [1] "57.65%"
```

The model successfully recovers a large change in κ caused by the treatment, alongside only a negligible

potential change in μ (similar posterior density to the left, 58%, and right of 0°). Since the model formula is identical to the change in direction example, we can extract the predictions in the same way.

```
draws_conc = as_draws_df(model_conc)

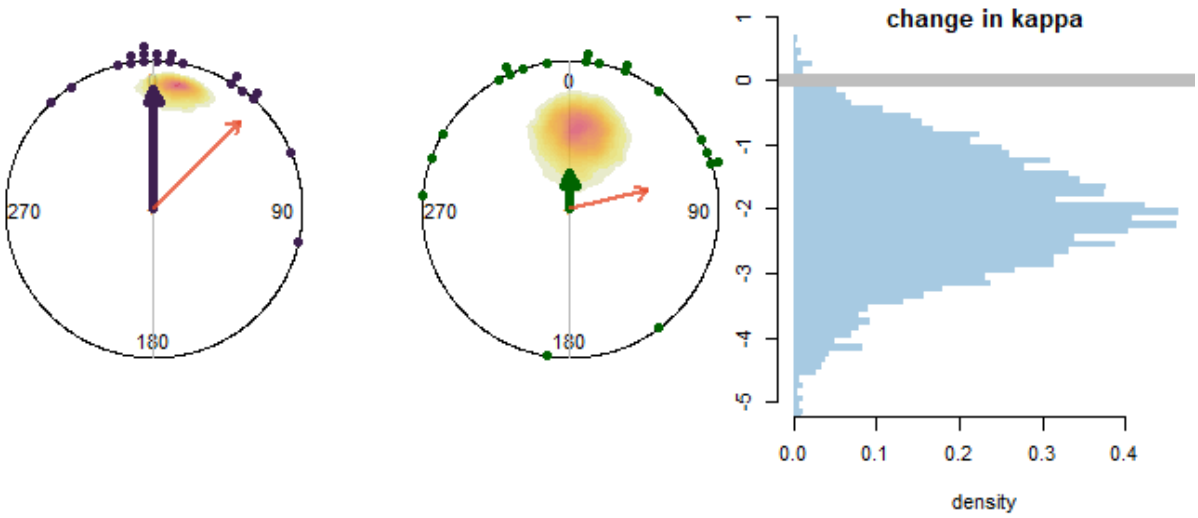
par(pty = 's')
par(mar = c(0,0,0,0),
    mfrow = c(1,3))
PCfun(cd_3,
      col = col_obs,
      sep = 0.05,
      shrink = 1.25,
      plot_rho = FALSE)
Draws2Cont(draws_conc,
           x_string = 'sin(b_Intercept)*A1(softplus(b_kappa_Intercept))',
           y_string = 'cos(b_Intercept)*A1(softplus(b_kappa_Intercept))',
)
arrows.circular(x = circular(0,
                             units = 'degrees',
                             rotation = 'clock',
                             zero = pi/2),
                y = A1(3),
                col = col_obs,
                lwd = 5,
                length = 0.1/1.25
)
with(draws_conc,
     arrows.circular(x = median.circular(x =
                             circular(b_Intercept,
                                  rotation = 'clock',
                                  zero = pi/2)
                                  ),
                     y = A1(softplus(x = median(x = b_kappa_Intercept))),
                     lwd = 2,
                     length = 0.1/1.25,
                     col = adjustcolor(col_sd, alpha.f = 200/255))
)

PCfun(cd_0.5,
      col = col_treat,
      sep = 0.05,
      shrink = 1.25,
      plot_rho = FALSE)
Draws2Cont(draws_conc,
           x_string = 'sin(b_Intercept+b_x)*A1(softplus(b_kappa_Intercept+b_kappa_x))',
           y_string = 'cos(b_Intercept+b_x)*A1(softplus(b_kappa_Intercept+b_kappa_x))',
)
arrows.circular(x = circular(0,
                             units = 'degrees',
                             rotation = 'clock',
                             zero = pi/2),
                y = A1(0.5),
                col = col_treat,
                lwd = 5,
```

```
               length = 0.1/1.25
)
with(draws_conc,
     arrows.circular(x = median.circular(x =
                              circular(b_Intercept + b_x,
                                   rotation = 'clock',
                                   zero = pi/2)
                                   ),
                     y = A1(softplus(x = median(x = b_kappa_Intercept +
                                                b_kappa_x))),
                     lwd = 2,
                     length = 0.1/1.25,
                     col = adjustcolor(col_sd, alpha.f = 200/255))
)
#nearly all estimates suggest a reduction in kappa
prop_decrease_conc = with(draws_conc, mean(b_kappa_x < 0))
paste0(prop_decrease_conc*100, '%')
```

```
## [1] "99.275%"
```

```
#calculate contrast on true scale by softplus transforming estimates
with(draws_conc,
     VertHist(data = softplus(b_kappa_Intercept+b_kappa_x) - softplus(b_kappa_Intercept),
              main = '\nchange in kappa',
              ylim = c(-5, 1),
              col = adjustcolor(col_kappa, alpha.f = 100/255),
              axes = FALSE,
              cex.axis = 0.7))
abline(h = 0,
       col = 'gray',
       lwd = 7)
axis(1)
axis(2, at = (-5:1))
```



Although there is some uncertainty in the particular size of the change in κ, 99.3% of posterior estimates support a treatment effect that decreases κ. *N.B.* Since κ is estimated on the inverse-softplus scale, the estimates for the control and treatment need to be calculated and then softplus transformed before a difference in κ can be calculated on its true scale.

Test the concentration change hypothesis

We can investigate this further with model comparison by fitting models with no effects of treatment, and effects on only kappa. We do this by setting the formula for μ and κ to `~ 0 + Intercept` (estimate intercept only).

```
#model with effects of treatment on kappa, but not mu
model_conc_kappa =  brm(formula = bf(y ~ 0 + Intercept,
                                     kappa ~ 0 + Intercept + x),
                        data = dt_conc,
                        family = unwrap_von_mises,
                        prior = prior_conc[c(1,3:4),], # the all kappa & 1st mu prior
                        stanvars = stan_unwrap_fun + modulo_circular_fun,
                        silent = 2,
                        backend = 'cmdstan') #model with no effects of treatment
model_conc_nokappa =  brm(formula = bf(y ~ 0 + Intercept,
                                       kappa ~ 0 + Intercept),
                          data = dt_conc,
                          family = unwrap_von_mises,
                          prior = prior_conc[c(1,3),], # intercept kappa & mu priors
                          stanvars = stan_unwrap_fun + modulo_circular_fun,
```

```
                        silent = 2,
                        backend = 'cmdstan' )
#all models converge well
print(   list(`treat. kappa` =      summary(model_conc_kappa),
              `unwrapped rhat` =
                UnwrapRhats(model_conc_kappa,
                            variable = c('b_Intercept'),
                            regex = FALSE
                                  ),
              `no treat.` =      summary(model_conc_nokappa),
              `unwrapped rhat no treat.` =
                UnwrapRhats(model_conc_nokappa,
                            variable = c('b_Intercept'),
                            regex = FALSE
                            )
              ),   digits = 3 )
```

```
## $`treat. kappa`
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ 0 + Intercept
##          kappa ~ 0 + Intercept + x
##    Data: dt_conc (Number of observations: 40)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS Tail_ESS
## Intercept          0.177     0.126   -0.073    0.431 1.001     2245     1925
## kappa_Intercept    3.374     0.919    1.768    5.285 1.003     1902     1835
## kappa_x           -2.341     1.063   -4.470   -0.381 1.002     1896     2243
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
##
## $`unwrapped rhat`
## b_Intercept
##           1
##
## $`no treat.`
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ 0 + Intercept
##          kappa ~ 0 + Intercept
##    Data: dt_conc (Number of observations: 40)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS Tail_ESS
## Intercept          0.165     0.136   -0.100    0.428 1.001     2611     2168
## kappa_Intercept    1.857     0.453    1.007    2.785 1.001     3114     2671
##
```

```
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
##
## $`unwrapped rhat no treat.`
## b_Intercept
##           1
```

```
loo_conc = loo(model_conc)
loo_conc_kappa = loo(model_conc_kappa)
loo_conc_nokappa = loo(model_conc_nokappa)
print(loo_compare(loo_conc, loo_conc_kappa, loo_conc_nokappa))
```

```
##                     elpd_diff se_diff
## model_conc_kappa     0.0       0.0
## model_conc          -1.0       0.4
## model_conc_nokappa  -1.9       2.5
```

In this case, model comparison is less conclusive than the posterior estimates. The model with an effect of treatment on κ only predicts the data better than the model with no effects of treatment, but this difference is small relative to its standard error (ELPD difference: 1.9 ± 2.5). With this sample size, cross-validation therefore provides only weak support for the change in concentration indicated by 99.3% of posterior estimates. As we might have also expected, the effect of treatment on μ, present in our original model but not the κ-only model, does not improve predictive density (ELPD difference: -1.0 ± 0.4), so there is little evidence that the treatment affects μ.

High inter-individual correlation

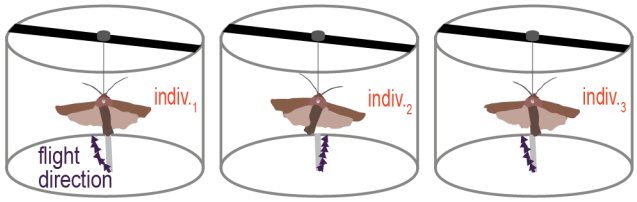


Figure 4: Tethered Bogong moths can be induced to fly in "flight simulators", which allows repeated measurements of flight directions across multiple individuals simultaneously. Individual directional biases may differ somewhat, but are typically highly correlated, aligning with the population's migratory direction.

In datasets that include repeated measurements across individuals, it is necessary to account for individual differences before attempting to identify any population level trends. In the simplest case, the orientation behaviour of different individuals is very similar. One example would be where all individuals share a common goal direction β_0, but differ in alignment with that direction by their individual offsets from that direction β_i. In this case μ and κ depend on the specific values of β_0 and β_i, and the distribution across β_i is determined by the hyperparameter κ_μ, the concentration across individual offsets.

In this example, each individual's headings are highly concentrated around their particular directional bias with the distribution $\mu_i = \beta_0 + \beta_i$, $\kappa = 5.0$. The distribution of individual biases is moderately concentrated, $\kappa_\mu = 2.0$, so that the heading biases of different individuals are quite close together.

```
kappa_mu_highcorr = 2.0 # concentration _across_ individuals
kappa_id_highcorr = 5.0 # concentration _within_ individual

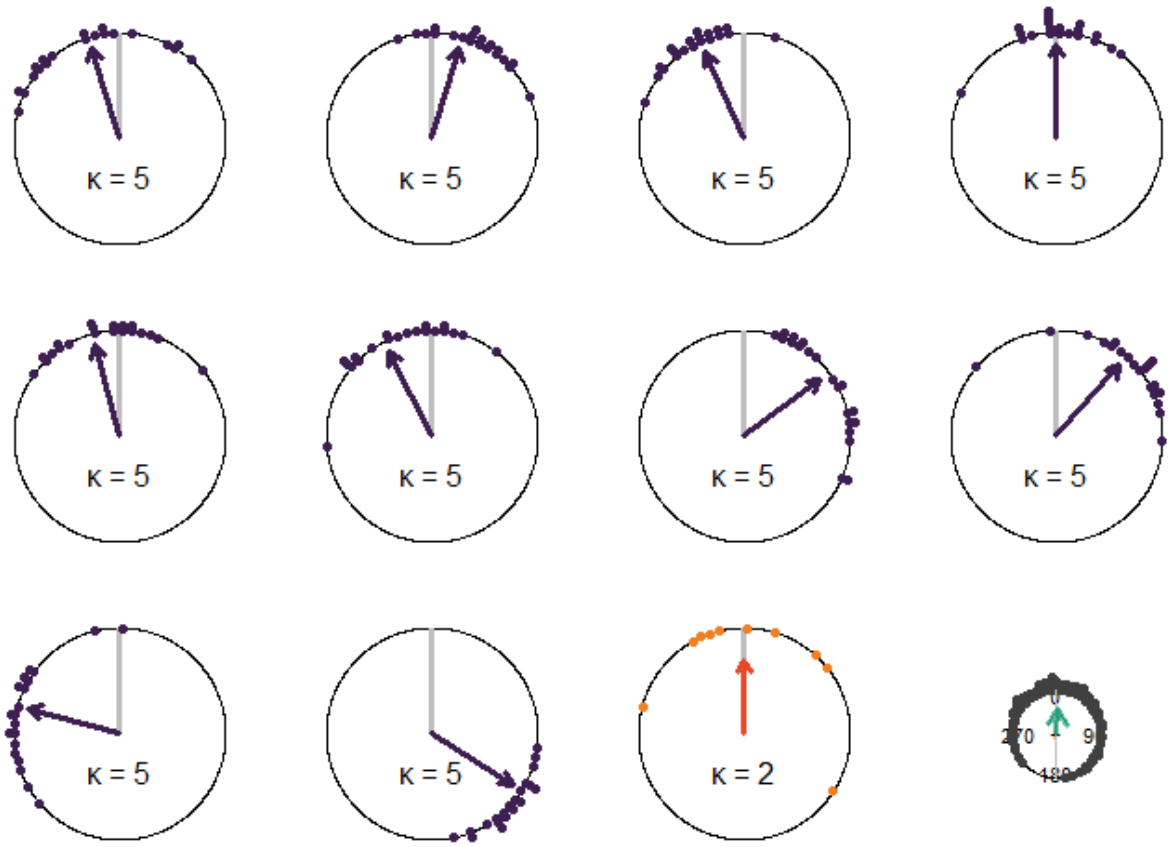
ndata = 10 # moderate sample size
set.seed(0120810506)#ISBN Batschelet, 1981
```

```r
mu_highcorr = rvonmises(n = 10,
                mu = c0,
                kappa = kappa_mu_highcorr)
print(round(mu_highcorr))
```

```
## Circular Data:
## Type = angles
## Units = degrees
## Template = none
## Modulo = asis
## Zero = 1.570796
## Rotation = clock
##  [1]  -18   16  -25    0  -16  -27   54   41  -74 -238
```

```r
par(pty = 's')
par(mar = c(0,0,0,0))
par(mfrow = c(3,4))
#Loop through individuals generating a random sample for each individual
id_highcorr = mapply(FUN = DescriptCplot,
                m = mu_highcorr,
                seed = 1000*1:length(mu_highcorr), #different seed for each
                save_sample = TRUE,
                k = kappa_id_highcorr,#all the same concentration
                ndata = 20,
                refline = 0,
                sdcol = NA,
                denscol = NA,
                mvcol = col_obs,
                lwd = 5,
                SIMPLIFY = FALSE
                )
#Add the population of biases
DescriptCplot(k = kappa_mu_highcorr,
              ndata = 10,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = col_sd,
              mvcol = col_sd
)
points.circular(mu_highcorr,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho
)
#Show the pooled data across all individuals
dt_comb_highcorr = do.call(what = c,
                   args = id_highcorr)
PCfun(angles = dt_comb_highcorr,
      col = 'gray25',
      shrink = 3.0)
```

```
#Fit the maximum likelihood distribution for the pooled data
mle_comb_highcorr = mle.vonmises(x = dt_comb_highcorr,bias = TRUE)
#Plot the estimated mean vector across pooled data
with(mle_comb_highcorr,
     {
       arrows.circular(x = circular(mu,
                                    units = 'degrees',
                                    rotation = 'clock',
                                    zero = pi/2),
                       y = A1(kappa),
                       lwd = 3,
                       col = col_pdf,
                       length = 0.1
       )
     }
)
```



The high correlation is apparent, since the majority of individual heading biases are within 45° of the population mean (orange points). Each individual's sample is also tightly clustered around its individual mean (dark purple points). Pooling data across individuals (grey points), we can see a preponderance of data in the population mean direction, but a much wider spread (lower concentration) than within any one individual's headings.

We can use a hierarchical model, accounting for differences and similarities between individuals, to measure concentration for the average individual (α_0) and across individual heading biases (κ_μ), as well as to confirm expectations about the population mean direction (β_0).

Fit the model

To model this in BRMS we can construct a 'nonlinear' formula for predicting the vector of angles y that is comprised of the sum estimates of β_0 and β_i. β_0 is estimated as an `Intercept` (single estimate across individuals), and β_i lacks an intercept (`~0`) but includes a different offset for each individual.

```
#collect data
dt_highcorr = data.frame(y = as.numeric( rad(unlist(id_highcorr)) ),#angles as a numeric vector in radi
                         ID = factor(x = #individual labels as an unordered categorical (factor)
                                       sort(
                                         rep(1:length(id_highcorr),
                                             times = length(id_highcorr[[1]]))
                                         ),
                                     ordered = FALSE)
                               )
#set up formula with both 'nonlinear' and linear predictors
form_highcorr = bf(y ~ mu,
                   nlf(mu ~ beta0 + betai),#a nonlinear formula
                   beta0 ~ 1,
                   betai ~ 0 + ID,
                   kappa ~ 1 + (1|ID),#a linear formula
                   family = unwrap_von_mises)
```

We can also estimate the concentration across β_i, by defining the hyperparameter `softkappamu` in `Stan` code, which is the inverse-softplus of estimates for the κ_μ parameter. This is linked to estimates of β_i because its softplus transform (`log1p_exp(softkappamu)`) acts as a prior on the von Mises distribution of β_i: $\beta_i \sim \mathcal{M}(0, \kappa_\mu)$. This parameter is not explicitly stated in our model formula, so its prior needs to be added as Stan code, using the `set_prior` function with `check = FALSE` to inform `BRMS` that is does not need to check the prior against the model formula. Since our κ_μ estimates need to balance similarity between individuals against any identifiable differences (partial pooling) we should set a prior distribution for κ_μ with high prior density towards strong correlation ($10 < \kappa_\mu < 20$) but some density towards lower values. A log-normal distribution $\log \kappa_\mu \sim \mathcal{N}(\log(15), 0.6)$ balances these two objectives.

To include individual differences in κ in the model, we can rely on the automatic methods for linear modelling in `BRMS`. That means that to the formula for population level κ (`kappa_Intercept`: `kappa~1`) we add the individual-effects term `+(1|ID)`. As for β_i, this implies a set of offsets from the population for each individual (α_i) that follow a normal distribution (on the inverse-softplus scale) defined by σ_κ. As for κ_μ we require a prior that balances correlation and potentially large differences, with high prior density for $\sigma_\kappa \approx 0$ but some density at much larger values. This can be achieved with a the positive half of a student-T distribution with 3 degrees of freedom, a mean at 0, and a standard deviation of 0.5: $\sigma_\kappa \sim \text{Half}\mathcal{T}(3, 0, 0.5)$.

To apply our von Mises prior to the vector of β_i estimates, we now also need to update our `unwrap_von_mises` function, to include a version for vectors (`unwrap_von_mises_vect`).

```
prior_highcorr =  prior('normal(0,3*pi()/2)', class = 'b', nlpar = 'beta0') +
                  prior('unwrap_von_mises_vect(0, log1p_exp(softkappamu))',
                        nlpar  = 'betai',  class = 'b') +
                  set_prior("target += lognormal_lpdf(log1p_exp(softkappamu) | log(15), 0.6)", #expect
                            check = FALSE) +
                  prior('normal(3,3)', class = 'Intercept', dpar = 'kappa') +
                  prior('student_t(3, 0, 0.5)', class = 'sd', dpar = 'kappa')

#Introduce stan variable for kappamu
stan_softkappamu = stanvar(scode = "
real softkappamu;
                           ",
                           block = "parameters") +
```

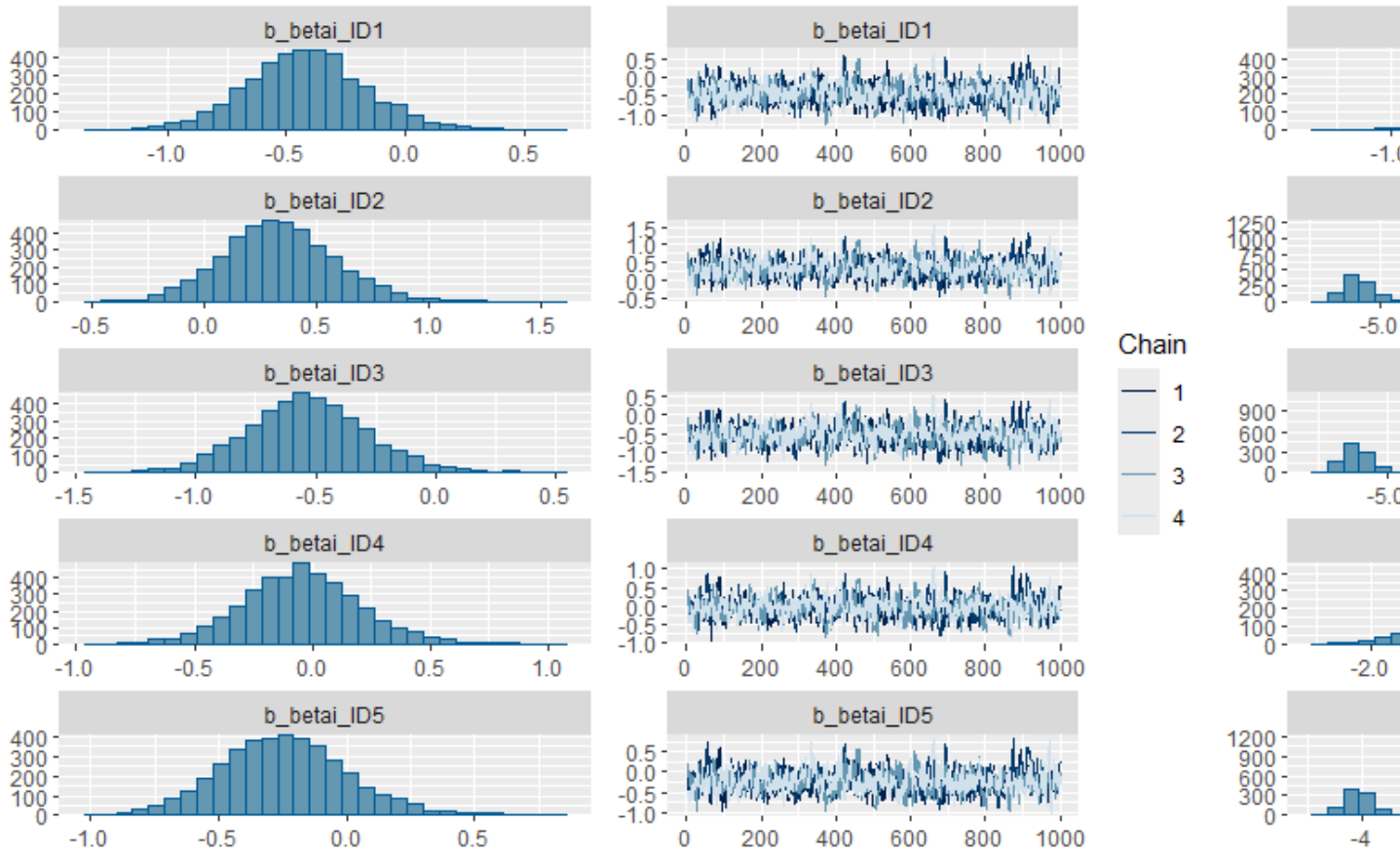
```
  stanvar(scode = "
real kappa_mu = log1p_exp(softkappamu);
          ",
          block = 'genquant')

#update the unwrap function to account for vectors
stan_unwrap_fun = stanvar(scode = "
    real unwrap_von_mises_lpdf(real y, real mu, real kappa) {
      return von_mises_lpdf(y | modulo_circular(mu), kappa);
    }
    real unwrap_von_mises_rng(real mu, real kappa) {
      return von_mises_rng( modulo_circular(mu) , kappa);
    }
    real unwrap_von_mises_vect_lpdf(vector y, real mu, real kappa) {
    real tmp = 0;
    for(i in 1:size(y))
    {
    tmp = tmp + unwrap_von_mises_lpdf(y[i] | mu, kappa);
    }
      return tmp;
    }
  ",
                          block = 'functions')

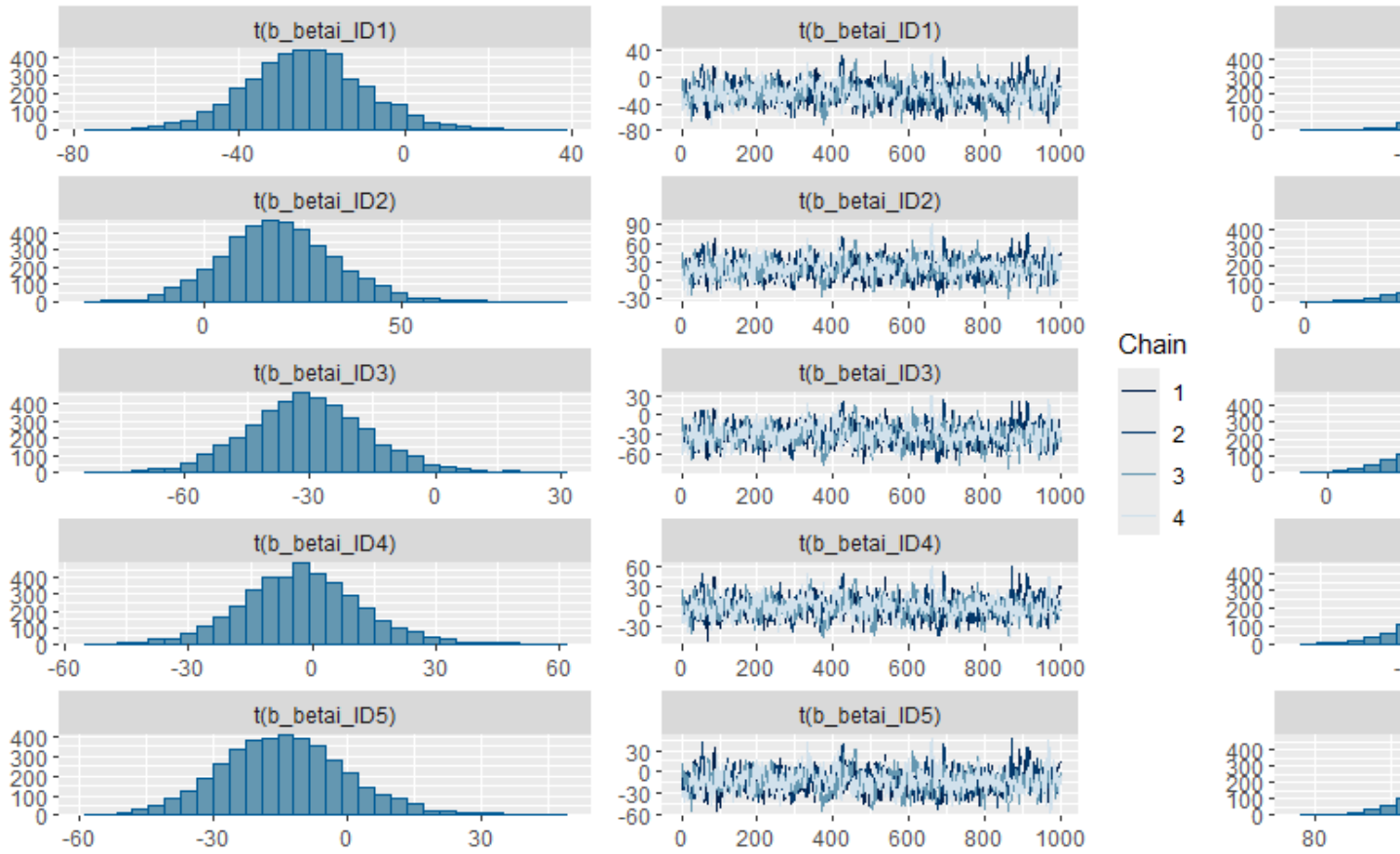

#run the model
model_highcorr = brm(
          formula = form_highcorr,
           data = dt_highcorr,
           family = unwrap_von_mises,
           stanvars = stan_unwrap_fun +
                      modulo_circular_fun +
                      stan_softkappamu,
           prior = prior_highcorr,
          cores = 4,
          silent = 2,
          backend = 'cmdstan')
#individuals for which raw estimates appear not to have converged (before unwrapping)
rhat_raw_highcorr = rhat(model_highcorr)[paste0('b_betai_ID', 1:length(id_highcorr))]
wrapped_highcorr = which(rhat_raw_highcorr > 1.05)
```

As for the previous models, for some of the circular variables estimates from different chains are separated (in this run, for individuals 7, 8, and 10), which can make it appear that the model has not converged. Each β_i estimate behaves in this way, and needs to be unwrapped before we can determine whether the chains have arrived at similar estimates.

```
#not all raw estimates look like they have converged
plot(model_highcorr,
      ask = FALSE,
      variable = 'betai',
      regex = TRUE) #plot raw estimates
```




```
#but they overlap when unwrapped
plot(x = model_highcorr,
     variable = c('betai'),
     regex = TRUE,
     ask = FALSE,
     transform = unwrap_circular_deg)
```



Calculating the $\hat{R}$ values for unwrapped estimates for β_i reveals that all estimates converged well ($\hat{R} < 1.01$).

```
sm_highcorr = summary(model_highcorr)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
print(sm_highcorr, digits = 2)
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 1
##          betai ~ 0 + ID
##          kappa ~ 1 + (1 | ID)
##    Data: dt_highcorr (Number of observations: 200)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 10)
##                     Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(kappa_Intercept)     0.55      0.47     0.02     1.74 1.00     1658     1752
##
## Regression Coefficients:
```

```
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_Intercept     5.03      0.54     4.04     6.15 1.00     3593     2536
## beta0_Intercept     0.05      0.24    -0.46     0.50 1.00      573      823
## betai_ID1          -0.42      0.26    -0.91     0.12 1.00      630     1040
## betai_ID2           0.33      0.25    -0.15     0.86 1.00      663      941
## betai_ID3          -0.53      0.25    -1.00     0.00 1.00      650      947
## betai_ID4          -0.04      0.25    -0.52     0.49 1.00      669      959
## betai_ID5          -0.24      0.25    -0.71     0.29 1.00      673      938
## betai_ID6          -0.35      0.26    -0.83     0.19 1.00      663      948
## betai_ID7          -0.65      2.75    -5.71     1.44 1.53        7       32
## betai_ID8          -0.83      2.74    -5.88     1.25 1.53        7       37
## betai_ID9          -1.37      0.26    -1.87    -0.82 1.00      672      908
## betai_ID10          0.67      2.75    -4.37     2.75 1.53        7       29
##
## Further Distributional Parameters:
##          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_mu     2.85      0.82     1.50     4.73 1.00     3331     2261
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
#beta0 and betai look bad, but good after unwrapping
UnwrapRhats(model_highcorr,
            variable = '^b_beta0')
```

```
## b_beta0_Intercept
##           0.99999
```

```
UnwrapRhats(model_highcorr,
            variable = '^b_betai')
```

```
##  b_betai_ID1  b_betai_ID2  b_betai_ID3  b_betai_ID4  b_betai_ID5  b_betai_ID6
##      0.99992      1.00112      1.00006      1.00002      1.00023      1.00010
##  b_betai_ID7  b_betai_ID8  b_betai_ID9 b_betai_ID10
##      0.99983      1.00053      0.99979      0.99977
```

The estimates of κ_μ here have a slight positive bias, due to a combination of low sample size and our prior to higher values. The model nonetheless recovers good estimates of β_0 and β_i. Importantly, this also allows us to estimate population κ, that is the concentration of the average individual, which needs to be distinguished from individual differences and the concentration across individuals.

```
draws_highcorr = as_draws_df(model_highcorr)

par(pty = 's')
par(mar = c(0,0,0,0),
    mfrow = c(3,4))
for(i in 1:length(id_highcorr) )
{
  mu_name = paste0('b_betai_ID',i)
  kappa_name = paste0('r_ID__kappa[',i,',Intercept]')
  PCfun(id_highcorr[[i]],
        col = col_obs,
        sep = 0.05,
        shrink = 1.25,
        plot_rho = FALSE)
```

```
  arrows.circular(x = mu_highcorr[i],
                  y = A1(kappa_id_highcorr),
                  col = col_obs,
                  lwd = 5,
                  length = 0.1/1.25
  )
  Draws2Cont(draws_highcorr,
             x_string = 'sin(b_beta0_Intercept + get(mu_name))*
                         A1(softplus(Intercept_kappa+get(kappa_name)))',
             y_string = 'cos(b_beta0_Intercept + get(mu_name))*
                         A1(softplus(Intercept_kappa+get(kappa_name)))'
             )
  with(draws_highcorr,
       arrows.circular(x = median.circular(
                             circular(x =
                             mod_circular(b_beta0_Intercept + get(mu_name)),
                                                  units = 'radians',
                                                  rotation = 'clock',
                                                  zero = pi/2)
                                          )[1],
                       y = A1(softplus(median(Intercept_kappa+get(kappa_name)))),
                       lwd = 2,
                       length = 0.1/1.25,
                       col = adjustcolor(col_sd, alpha.f = 200/255))
  )
}

#Add the population of biases
DescriptCplot(k = kappa_mu_highcorr,
              ndata = 10,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = NA,
              mvcol = col_rho
)
points.circular(mu_highcorr,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho
)
Draws2Cont(draws = draws_highcorr,
           x_string = 'sin(b_beta0_Intercept)*
             A1(kappa_mu)',
           y_string = 'cos(b_beta0_Intercept)*
             A1(kappa_mu)'
           )

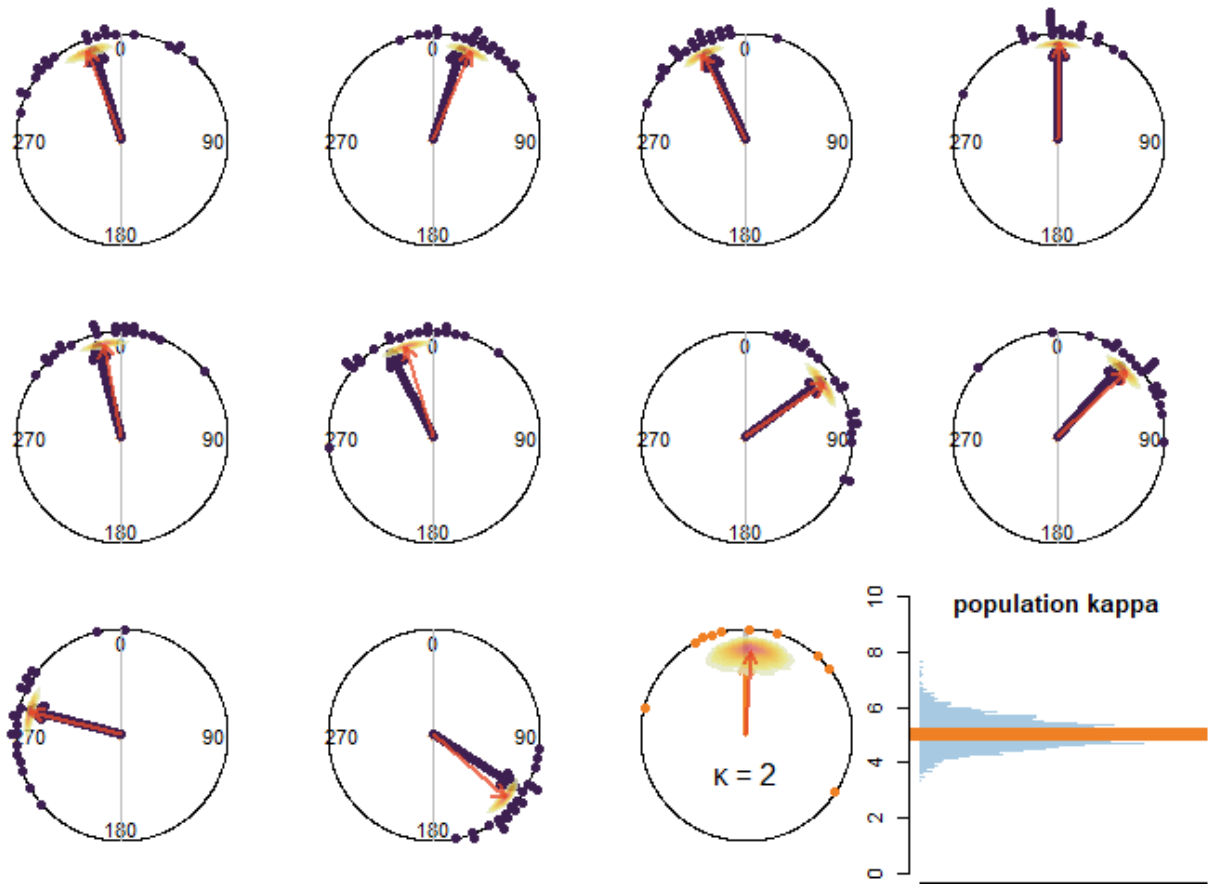
with(draws_highcorr,
     arrows.circular(x = mean.circular(circular(b_beta0_Intercept,
```

```
                                                    units = 'radians',
                                                    rotation = 'clock',
                                                    zero = pi/2)
     )[1],
     y = A1(median(kappa_mu)),
     lwd = 2,
     length = 0.1/1.25,
     col = adjustcolor(col_sd, alpha.f = 200/255))
)

with(draws_highcorr, paste0(mean(softplus(Intercept_kappa) > kappa_id_highcorr)*100, '%') ) #partial po
```

```
## [1] "51.3%"
```

```
with(draws_highcorr,
     VertHist(data = softplus(Intercept_kappa),
              main = '\npopulation kappa',
              ylim = c(0, 10),
              col = adjustcolor(col_kappa, alpha.f = 100/255),
              axes = FALSE,
              cex.axis = 0.7))
abline(h = kappa_id_highcorr,
       col = col_rho,
       lwd = 7)
axis(1)
axis(2, at = 0:5*2)
```



Despite the bias towards high concentration for κ_μ, all individual biases are recovered well by this model (overlap of shaded regions with dark purple arrows), including those more distant from the population mean direction.

Low Inter-individual correlation

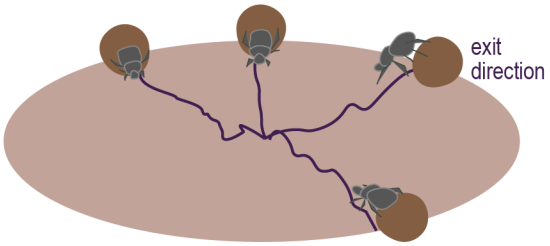


Figure 5: When rolling a dung ball, ball-rolling dung beetles can be repeatedly returned to the centre of a circular arena on exiting, resulting in repeated measurements of exit direction. While a single individual exits consistently in the same direction, there is typically no relationship between individual directional biases.

Individuals within the same population do not necessarily share similar directional biases, and where they do this relationship may be weak (low correlation). In this example each individual's headings have a high concentration ($\kappa = 5.0$), but the concentration across individual directional biases is low ($\kappa_\mu = 0.1$). As before, the mean direction of the generative distribution is at 0°, but, due to the greater probability of observations distant from this mean direction, this is not necessarily reflected by the mean of individuals in our sample.

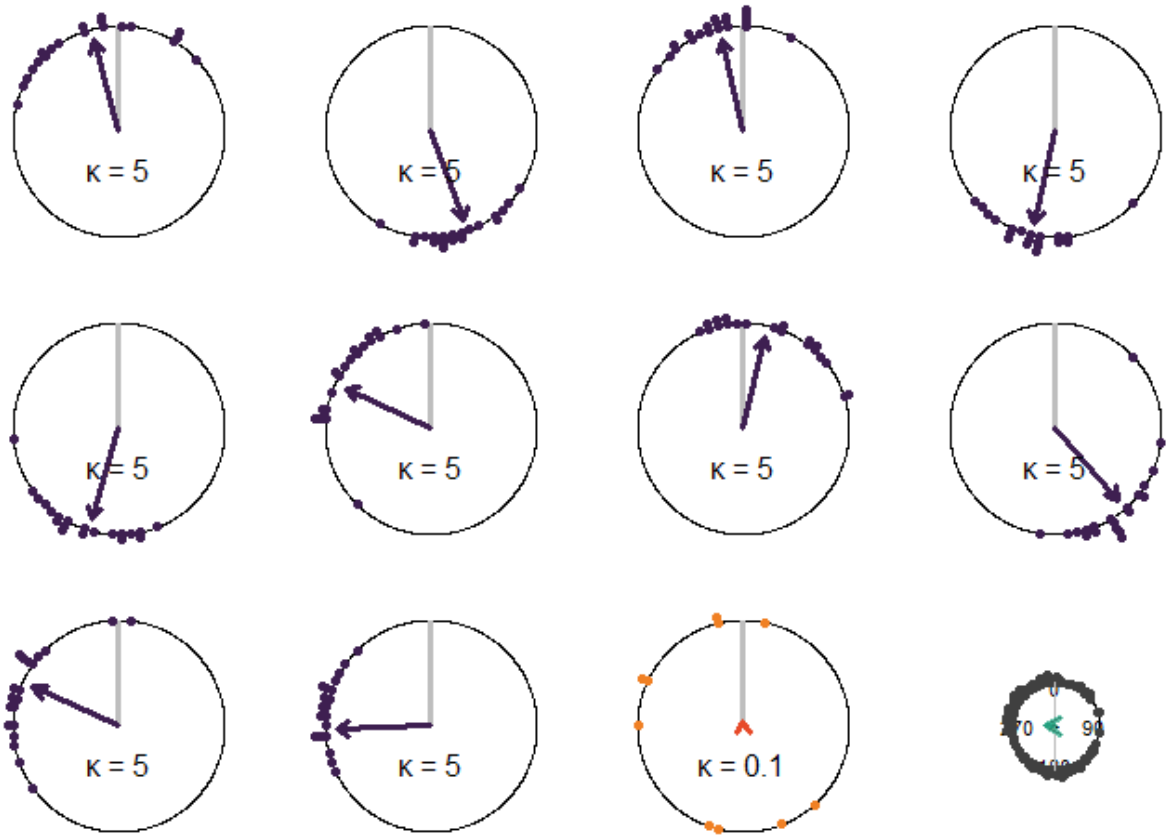
```
kappa_mu_lowcorr = 0.1 #very low concentration across individual heading biases
kappa_id_lowcorr = 5.0 #high within-individual concentration

set.seed(591005598)#DOI Baird et al., 2010
mu_lowcorr = rvonmises(n = ndata,
                mu = c0,
                kappa = kappa_mu_lowcorr)

par(pty = 's')
par(mar = c(0,0,0,0))
par(mfrow = c(3,4))
id_lowcorr = mapply(FUN = DescriptCplot,
                m = mu_lowcorr, #
                seed = 1000*1:length(mu_lowcorr), # different seed for each
                save_sample = TRUE,
                k = kappa_id_lowcorr,
                ndata = 20,
                refline = 0,
                sdcol = NA,
                denscol = NA,
                mvcol = col_obs,
                lwd = 5,
                SIMPLIFY = FALSE)
#Add the population of biases
DescriptCplot(k = kappa_mu_lowcorr,
              ndata = 10,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              mvcol = col_sd
)
points.circular(mu_lowcorr,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho
)

dt_comb_lowcorr = do.call(what = c,
                  args = id_lowcorr)
PCfun(angles = dt_comb_lowcorr,
      col = 'gray25',
      shrink = 3.0)
mle_comb_lowcorr = mle.vonmises(x = dt_comb_lowcorr,bias = TRUE)
with(mle_comb_lowcorr,
     {
       arrows.circular(x = circular(mu,
                                    units = 'degrees',
                                    rotation = 'clock',
                                    zero = pi/2),
                       y = A1(kappa),
```

```
                        lwd = 3,
                        col = col_pdf,
                        length = 0.1
        )
    }
)
```



Fit the model

Although the individual heading biases in this dataset are more variable than those in the high-correlation dataset, the underlying structure is the same. The same formula can account for large individual differences in heading just as well as small ones.

```
#collect data
dt_lowcorr = data.frame(y = as.numeric( rad(unlist(id_lowcorr)) ),#angles as a numeric vector in radian
                        ID = factor(x = #individual labels as an unordered categorical (factor)
                                    sort(
                                      rep(1:length(id_lowcorr),
                                          times = length(id_lowcorr[[1]]))
                                      ),
                                    ordered = FALSE)
                                )
#set up formula with both 'nonlinear' and linear predictors
form_lowcorr = bf(y ~ mu,
                  nlf(mu ~ beta0 + betai),#a nonlinear formula
```

```
                  beta0 ~ 1,
                  betai ~ 0 + ID,
                  kappa ~ 1 + (1|ID),#a linear formula
                  family = unwrap_von_mises)
```

Since the expectations regarding μ, κ and σ_κ do not differ from the high-correlation model, we can use the same priors for these parameters. As before, the prior for κ_μ accounts for potentially large differences in β_i, which in this case are a feature of the data, but the principle of partial pooling still necessitates a positive bias in our prior for κ_μ, to ensure any similarities across heading biases are captured by the model.

```
prior_lowcorr =  prior('normal(0,3*pi()/2)', class = 'b', nlpar = 'beta0') +
                 prior('unwrap_von_mises_vect(0, log1p_exp(softkappamu))',
                       nlpar  = 'betai',  class = 'b') +
                 set_prior("target += lognormal_lpdf(log1p_exp(softkappamu) | log(15), 0.6)", #expect
                           check = FALSE) +
                 prior('normal(3,3)', class = 'Intercept', dpar = 'kappa') +
                 prior('student_t(3, 0, 0.5)', class = 'sd', dpar = 'kappa')


#run the model
model_lowcorr = brm(
          formula = form_lowcorr,
           data = dt_lowcorr,
           family = unwrap_von_mises,
           stanvars = stan_unwrap_fun +
                      modulo_circular_fun +
                      stan_softkappamu,
           prior = prior_lowcorr,
           silent = 2,
          cores = 4,
          backend = 'cmdstan')
```

As for the previous models, for some of the circular variables estimates for different chains are separated, which can make it appear that the model has not converged. Each β_i estimate behaves in this way, and needs to be unwrapped before we can determine whether the chains have arrived at similar estimates.

```
sm_lowcorr = summary(model_lowcorr)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
print(sm_lowcorr, digits = 2)
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 1
##          betai ~ 0 + ID
##          kappa ~ 1 + (1 | ID)
##    Data: dt_lowcorr (Number of observations: 200)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
```

```
## ~ID (Number of levels: 10)
##                       Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(kappa_Intercept)       0.56      0.46     0.02     1.73 1.00     1884     1766
##
## Regression Coefficients:
##                  Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_Intercept      5.04      0.54     4.05     6.17 1.00     3863     2275
## beta0_Intercept     -0.82      4.54   -13.82     5.44 1.99        5       11
## betai_ID1            0.46      4.54    -5.80    13.46 1.99        5       11
## betai_ID2            0.58      4.78    -8.51    10.46 2.13        5       11
## betai_ID3            0.52      4.54    -5.74    13.50 1.99        5       11
## betai_ID4            2.65      4.74    -8.00    10.97 1.66        6       11
## betai_ID5            1.20      4.78    -7.88    11.06 2.12        5       11
## betai_ID6            1.42      4.28    -6.09    12.86 1.98        5       11
## betai_ID7            2.66      4.28    -4.87    14.07 1.98        5       11
## betai_ID8            3.34      4.54    -2.95    16.33 1.99        5       11
## betai_ID9            1.19      4.28    -6.33    12.65 1.98        5       11
## betai_ID10           0.97      4.28    -6.54    12.45 1.99        5       11
##
## Further Distributional Parameters:
##          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_mu     1.61      0.46     0.85     2.62 1.00     4139     1979
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
#betai looks bad, but good after unwrapping
#beta0 and betai look bad, but good after unwrapping
UnwrapRhats(model_lowcorr,
            variable = '^b_beta',#any variable starting "b_beta..."
            regex = TRUE)
```

```
## b_beta0_Intercept       b_betai_ID1       b_betai_ID2       b_betai_ID3
##           1.00169           1.00148           1.00180           1.00187
##       b_betai_ID4       b_betai_ID5       b_betai_ID6       b_betai_ID7
##           1.00136           1.00143           1.00236           1.00190
##       b_betai_ID8       b_betai_ID9      b_betai_ID10
##           1.00214           1.00136           1.00182
```

As for the high-correlation example, the model overestimates κ_μ , but recovers accurate estimates of β_i and population κ. As before, most posterior density is close to the true values of β_i and falls symmetrically around the true population κ. Perhaps unsurprisingly given the set of individual heading biases, the population mean direction β_0 was not successfully recovered, but the broad credible interval around the estimate would suggest there is considerable uncertainty in the model about the precise true direction. This may lead us to ask whether or not there is a shared population mean direction.

```
draws_lowcorr = as_draws_df(model_lowcorr)

par(pty = 's')
par(mar = c(0,0,0,0),
    mfrow = c(3,4))
for(i in 1:length(id_lowcorr) )
{
  mu_name = paste0('b_betai_ID',i)
  kappa_name = paste0('r_ID__kappa[',i,',Intercept]')
```

```
  PCfun(id_lowcorr[[i]],
        col = col_obs,
        sep = 0.05,
        shrink = 1.25,
        plot_rho = FALSE)
  arrows.circular(x = mu_lowcorr[i],
                  y = A1(kappa_id_lowcorr),
                  col = col_obs,
                  lwd = 5,
                  length = 0.1/1.25
  )
  Draws2Cont(draws_lowcorr,
             x_string = 'sin(b_beta0_Intercept + get(mu_name))*
                         A1(softplus(Intercept_kappa+get(kappa_name)))',
             y_string = 'cos(b_beta0_Intercept + get(mu_name))*
                         A1(softplus(Intercept_kappa+get(kappa_name)))'
             )
  with(draws_lowcorr,
       arrows.circular(x = median.circular(
                            circular(x =
                            mod_circular(b_beta0_Intercept + get(mu_name)),
                                                  units = 'radians',
                                                  rotation = 'clock',
                                                  zero = pi/2)
                                              )[1],
                       y = A1(softplus(median(Intercept_kappa+get(kappa_name)))),
                       lwd = 2,
                       length = 0.1/1.25,
                       col = adjustcolor(col_sd, alpha.f = 200/255))
  )
}

#Add the population of biases
DescriptCplot(k = kappa_mu_lowcorr,
              ndata = 10,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = NA,
              mvcol = col_rho
)
points.circular(mu_lowcorr,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho
)
Draws2Cont(draws = draws_lowcorr,
           x_string = 'sin(b_beta0_Intercept)*
             A1(kappa_mu)',
           y_string = 'cos(b_beta0_Intercept)*
```

```
            A1(kappa_mu)'
          )

with(draws_lowcorr,
     arrows.circular(x = mean.circular(circular(b_beta0_Intercept,
                                                units = 'radians',
                                                rotation = 'clock',
                                                zero = pi/2)
     )[1],
     y = A1(median(kappa_mu)),
     lwd = 2,
     length = 0.1/1.25,
     col = adjustcolor(col_sd, alpha.f = 200/255))
)
#posterior distribution of mean direction estimates: median and 95% CI (degrees clockwise from 0°)
ci_beta0_lowcorr = with(draws_lowcorr,
                        quantile(unwrap_circular_deg(b_beta0_Intercept),
                                 probs = c(0.025, 0.5, 0.975)) )
round(ci_beta0_lowcorr)
```

```
##  2.5%   50% 97.5%
##  -145   -85   -24
```

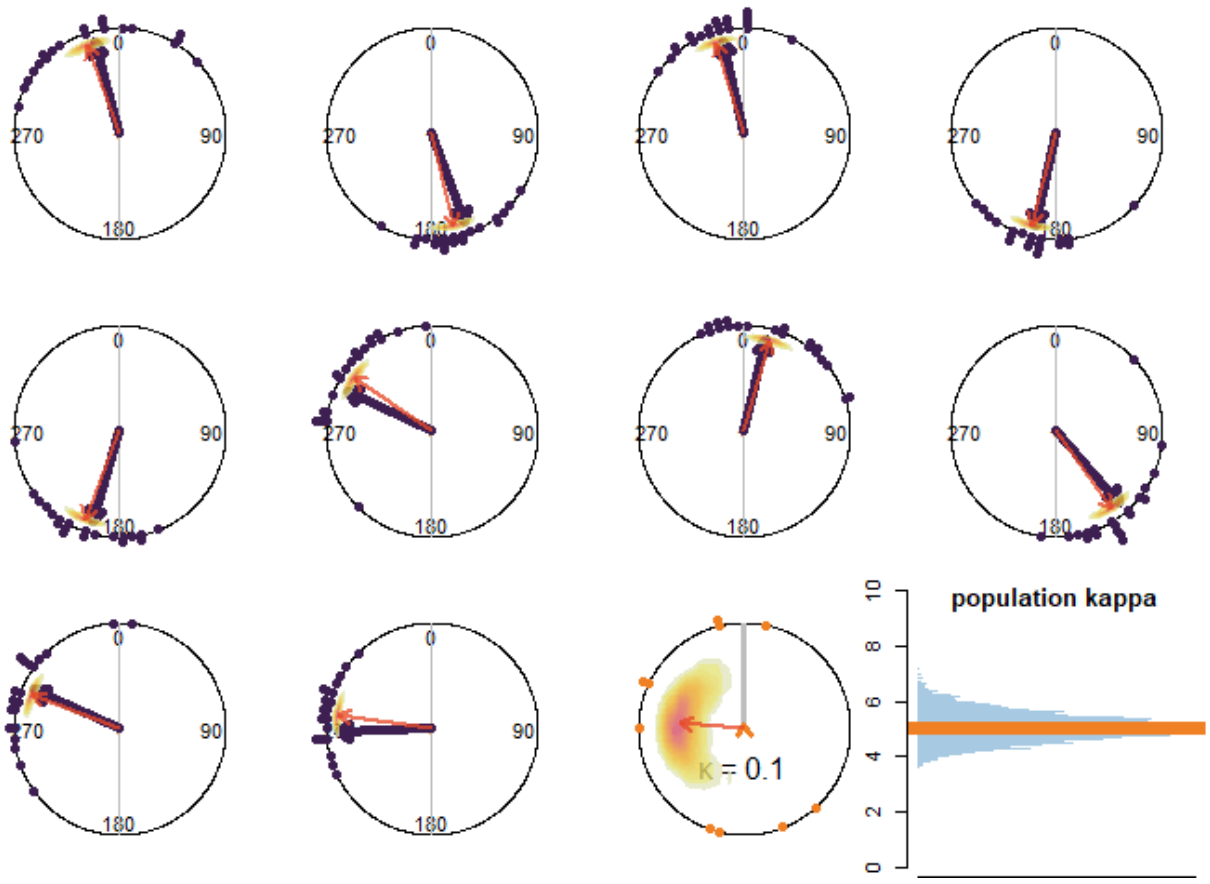
```
#width of the 95% CI
paste0(round(diff(ci_beta0_lowcorr[c(1, 3)])), '°')
```

```
## [1] "121°"
```

```
#posterior estimate of population kappa
with(draws_lowcorr, paste0(mean(softplus(Intercept_kappa) > kappa_id_lowcorr)*100, '%') ) #partial pool
```

```
## [1] "52.15%"
```

```
with(draws_lowcorr,
     VertHist(data = softplus(Intercept_kappa),
              main = '\npopulation kappa',
              ylim = c(0, 10),
              col = adjustcolor(col_kappa, alpha.f = 100/255),
              axes = FALSE,
              cex.axis = 0.7))
abline(h = kappa_id_lowcorr,
       col = col_rho,
       lwd = 7)
axis(1)
axis(2, at = (0:5)*2)
```



Test shared heading bias hypothesis

For this dataset, we might reasonably question whether individual heading biases have any shared bias. We can investigate this by fitting a model without a κ_μ parameter, but still accounting for individual heading biases and partial pooling for κ. We can set β_0 to 0° by assigning the formula `~0` (we could also exclude it entirely: `mu ~ betai`). Since there are no constraints on where β_i can be for each individual, we can set a (near) uniform von Mises prior with $\kappa \approx 0$, although a more complex model could benefit from the normal prior used for β_0 above. Without the extra information from κ_μ about the expected distribution of β_i sampling can become unstable, so we can use smaller sampling steps to avoid divergent transitions (which could impede sampling) by setting `adapt_delta` closer to 1.00 (default value 0.8). This slows down model estimation, but improves estimate reliability.

```
#model with effects of treatment on kappa, but not mu
#set up formula with both 'nonlinear' and linear predictors
form_nocorr = bf(y ~ mu,
                 nlf(mu ~ beta0 + betai),#a nonlinear formula
                 beta0 ~ 0, #not estimated, anchored at 0°
                 betai ~ 0 + ID,
                 kappa ~ 1 + (1|ID),#a linear formula
                 family = unwrap_von_mises)

prior_nocorr =  prior('unwrap_von_mises_vect(0.0,1e-16)', class = 'b', nlpar = 'betai') + #biases free
                 prior('normal(3,3)', class = 'Intercept', dpar = 'kappa') +
                 prior('student_t(3, 0, 0.5)', class = 'sd', dpar = 'kappa')
```

```r
#run the model
model_nocorr = brm(
          formula = form_nocorr,
           data = dt_lowcorr,
           family = unwrap_von_mises,
           stanvars = stan_unwrap_fun +
                      modulo_circular_fun,
           prior = prior_nocorr,
           silent = 2,
          cores = 4,
          control = list(adapt_delta = 0.97),#slower, but more robust sampling
          backend = 'cmdstan')

#all models converge well
print(   list(`no kappa_mu` =      summary(model_nocorr),
              `unwrapped rhat` =
                UnwrapRhats(model_nocorr,
                            variable = c('^b_beta')
                                 )
              ),   digits = 3 )
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
## $`no kappa_mu`
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 0
##          betai ~ 0 + ID
##          kappa ~ 1 + (1 | ID)
##    Data: dt_lowcorr (Number of observations: 200)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 10)
##                     Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS
## sd(kappa_Intercept)    0.546     0.447    0.017    1.641 1.003     1525
##                     Tail_ESS
## sd(kappa_Intercept)     1274
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI  Rhat Bulk_ESS Tail_ESS
## kappa_Intercept    5.055     0.530    4.086    6.169 1.001     2723     2059
## betai_ID1         -0.341     0.114   -0.562   -0.126 1.001     4401     3128
## betai_ID2         -0.252     3.144   -3.555    3.063 1.734        6      158
## betai_ID3         -0.290     0.100   -0.485   -0.097 1.001     5414     3141
## betai_ID4          0.250     3.144   -3.048    3.558 1.734        6      115
## betai_ID5         -1.205     2.723   -2.973    3.640 1.528        7       32
## betai_ID6         -0.971     0.108   -1.187   -0.760 1.001     4024     3032
## betai_ID7          0.288     0.112    0.073    0.507 1.001     4617     2745
```

```
## betai_ID8          0.935     2.724   -3.921    2.698 1.528        7       31
## betai_ID9          0.366     2.721   -1.392    5.217 1.528        7       30
## betai_ID10        -1.416     0.101   -1.616   -1.215 1.001     4725     2867
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
##
## $`unwrapped rhat`
##  b_betai_ID1  b_betai_ID2  b_betai_ID3  b_betai_ID4  b_betai_ID5  b_betai_ID6
##            1            1            1            1            1            1
##  b_betai_ID7  b_betai_ID8  b_betai_ID9 b_betai_ID10
##            1            1            1            1
```

```
loo_lowcorr = loo(model_lowcorr)
loo_nocorr = loo(model_nocorr)
print(loo_compare(loo_lowcorr, loo_nocorr))
```

```
##               elpd_diff se_diff
## model_nocorr   0.0       0.0
## model_lowcorr -0.1       0.4
```

Although technically this dataset was simulated with a von Mises distribution with very low concentration $\kappa_\mu = 0.1$, the distribution of individual headings does not detectably differ from uniform. This is certainly plausible for animals without a shared goal direction, and at this level of concentration a considerably larger number of individuals would be needed to recover any group-level bias. Nonetheless, both models recover the concentration of the average individual's headings ($\kappa = \log(\exp(5.0) + 1) \approx 5.0$).

Variable individual parameters

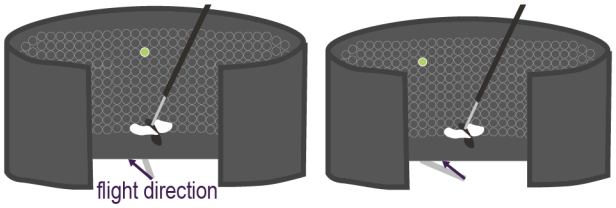


Figure 6: Tethered vinegar flies (*Drosophila*) can be induced to fly in a "virtual reality" LED arena, in which their wing motion causes visual cues to rotate. A green light spot induces orientation behaviour, with a weak population level bias towards the light, but highly variable performance across individuals.

In most datasets, we can reasonably expect inter-individual variation for both μ and κ: individuals may differ in perceived or genuine goal direction, accuracy and consistency. In this example, there is an moderate concentration across individual biases ($\kappa_\mu = 1.0$), and the average individual's headings are somewhat concentrated ($\alpha_0 = 1.5$). However, the concentration within each individual's headings varies ($\sigma_\kappa = 0.5$) with a standard deviation 1/3 of the mean. This introduces the possibility of large variations in individual concentration (α_i), including a $\approx 0.1\%$ chance of completely uniform heading distributions ($\kappa = 0$) for some individuals (assuming a truncated normal distribution).

```
kappa_mu_var = 1.0 # concentration across individuals
kappa_var_mean = 1.5 #concentration of the average individual
kappa_var_sd = 0.5 #standard deviation of concentration across individuals
#simulate data
set.seed(0120810506)#ISBN Batschelet, 1981
```

```
kappa_id_var = rnorm(n = ndata,
                     mean = kappa_var_mean,
                     sd = kappa_var_sd)
#rectified
kappa_id_var[kappa_id_var<0] = 0

set.seed(0120810506)#ISBN Batschelet, 1981
# list of circular datasets
mu_var = rvonmises(n = ndata, mu = c0, kappa = kappa_mu_var)

par(pty = 's')
par(mar = c(0,0,0,0))
par(mfrow = c(3,4))
#generate a list of individuals with different heading biases
#and different concentrations
id_var = mapply(m = mu_var,
                k = round(kappa_id_var,2),
                seed = 10001+1:length(mu_var), # different seed for each
                FUN = DescriptCplot,
                save_sample = TRUE,
                ndata = 40,
                refline = 0,
                sdcol = NA,
                denscol = NA,
                mvcol = col_obs,
                lwd = 5,
                SIMPLIFY = FALSE)
#Add the population of biases
DescriptCplot(k = kappa_mu_var,
              ndata = ndata,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = col_sd,
              mvcol = col_sd )

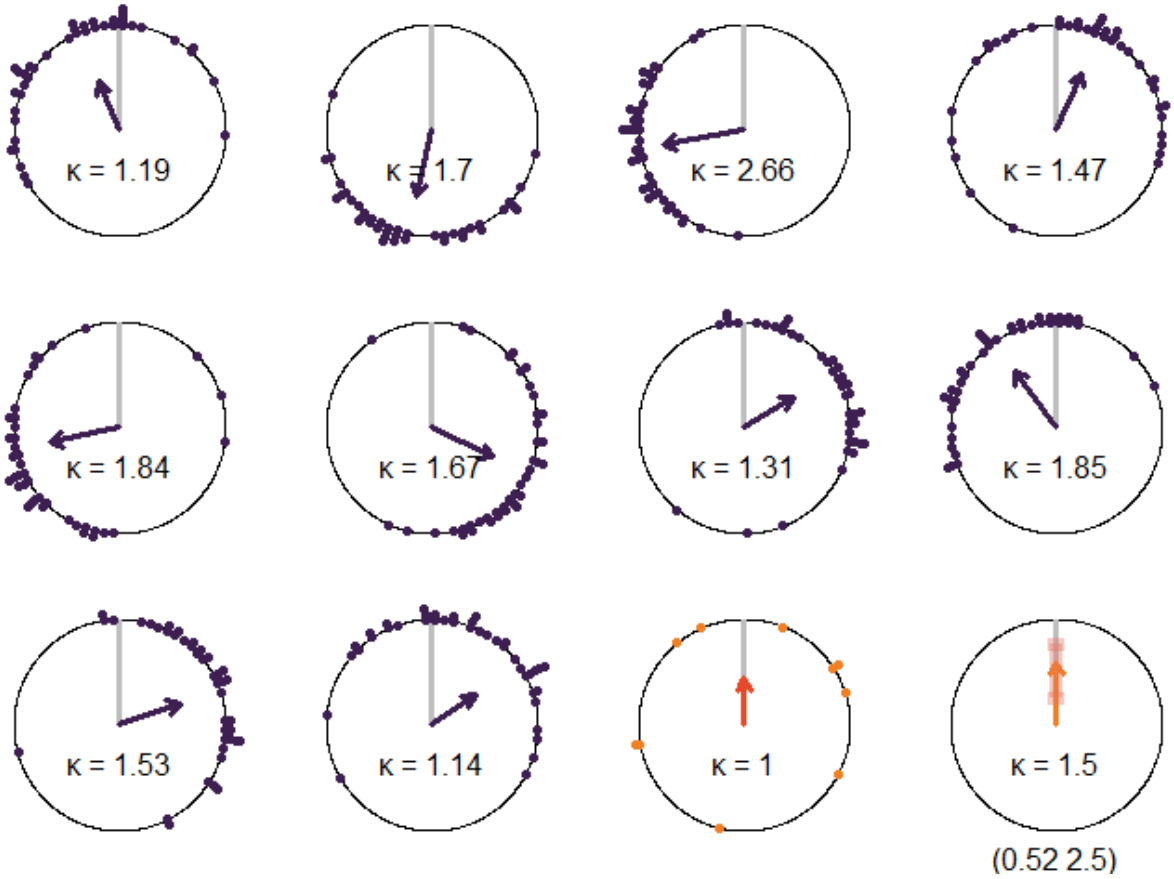
points.circular(mu_var,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho )

mean_kappa_id_var = softplus(mean(inv_softplus(kappa_id_var)))
#Add decription of the average individual
DescriptCplot(k = kappa_var_mean,
              ndata = ndata,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = col_rho,
```

```
              mvcol = col_rho )
kappa_id_var_ci = kappa_var_mean +
                  kappa_var_sd*qnorm(c(0,1) + c(1,-1)*0.05/2)
#rectify
kappa_id_var_ci[kappa_id_var_ci<0] = 0

arrows(x0 = sin(c0),
       x1 = sin(c0),
       y0 = A1(kappa_id_var_ci[1]),
       y1 = A1(kappa_id_var_ci[2]),
       lwd = 7,
       col = adjustcolor(col = col_sd2, alpha.f = 100/255),
       length = 0.05,
       angle = 90,
       code = 3,
       lend = 'butt' )

mtext(text = paste0('(',paste(signif(kappa_id_var_ci, 2), collapse = ' '), ')'), side = 1, line = -1)
```



Despite the modest inter-individual correlation, 4 individuals diverge from the population mean direction by $> 90°$. Individual concentrations also vary from $\kappa = 1.14$ to $\kappa = 2.66$, indeed in a larger population we would expect 95% of individual α_i in (0.5, 2.5), corresponding to (circular) standard deviations between 96° and 42°.

Fit the model

Although the individuals in this dataset are more variable than those in the high-correlation dataset, the underlying structure is the same. Since individuals have the potential to differ in both μ and κ, we can use the same formula as before, which already accounts for this.

```
#collect data
dt_var = data.frame(y = as.numeric( rad(unlist(id_var)) ),#angles as a numeric vector in radians
                          ID = factor(x = #individual labels as an unordered categorical (factor)
                                        sort(
                                          rep(1:length(id_var),
                                              times = length(id_var[[1]]))
                                          ),
                                      ordered = FALSE)
                               )
#set up formula with both 'nonlinear' and linear predictors
form_var = bf(y ~ mu,
                  nlf(mu ~ beta0 + betai),#a nonlinear formula
                  beta0 ~ 1,
                  betai ~ 0 + ID,
                  kappa ~ 1 + (1|ID),#a linear formula
                  family = unwrap_von_mises)
```

As before, the prior for κ_μ accounts for potentially large differences in β_i. Unlike the high-correlation dataset, it would be prudent to account for potentially larger σ_κ, while retaining high density for $\sigma_\kappa \approx 0$. This can be achieved by increasing the standard deviation of its prior to 1.0: $\sigma_\kappa \sim \text{Half}\mathcal{T}(3, 0, 1.0)$. In this case, we expect low average concentration, so we use the prior $\alpha_0 \sim \mathcal{N}(1, 3)$, which places more density in ($\kappa \approx 0.3$–3). If we are interested in recovering the population mean direction, we can use an unbiased von Mises prior ($\beta_0 \sim \mathcal{M}(0°, \approx 0)$) for mean direction.

```
prior_var =  prior('unwrap_von_mises_vect(0,1e-16)', class = 'b', nlpar = 'beta0') +
                  prior('unwrap_von_mises_vect(0, log1p_exp(softkappamu))',
                        nlpar  = 'betai',  class = 'b') +
                  set_prior("target += lognormal_lpdf(log1p_exp(softkappamu) | log(15), 0.6)", #expect
                            check = FALSE) +
                  prior('normal(1,3)', class = 'Intercept', dpar = 'kappa') +
                  prior('student_t(3, 0, 1.0)', class = 'sd', dpar = 'kappa')


#run the model
model_var = brm(
          formula = form_var,
           data = dt_var,
           family = unwrap_von_mises,
           stanvars = stan_unwrap_fun +
                      modulo_circular_fun +
                      stan_softkappamu,
           prior = prior_var,
           silent = 2,
          cores = 4,
          backend = 'cmdstan')
```

```
## Warning: 4 of 4000 (0.0%) transitions hit the maximum treedepth limit of 10.
## See https://mc-stan.org/misc/warnings for details.
```

```r
#number of transitions that reached the maximum tree depth (default 10)
n_treedepth_var = sum(subset(nuts_params(model_var), Parameter == 'treedepth__')$Value >= 10)
```

If `brms` warns that some transitions reached the maximum tree depth (4 of 4000 in this run), sampling was less efficient for those transitions. This does not bias the estimates, and can be avoided by increasing `max_treedepth` via the `control` argument.

As for the previous models, some of the circular variable estimates are separated between different chains, which can make it appear that the model has not converged. Each β estimate behaves in this way, and needs to be unwrapped before we can determine whether the chains have arrived at similar estimates.

```r
sm_var = summary(model_var)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```r
print(sm_var, digits = 2)
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 1
##          betai ~ 0 + ID
##          kappa ~ 1 + (1 | ID)
##    Data: dt_var (Number of observations: 400)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 10)
##                     Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(kappa_Intercept)     0.32      0.22     0.02     0.84 1.00     1358     1888
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_Intercept     1.87      0.20     1.51     2.28 1.00     2449     1916
## beta0_Intercept   -12.61     18.54   -38.85    25.08 2.25        5       12
## betai_ID1          12.04     18.54   -25.66    38.28 2.25        5       12
## betai_ID2          12.88     18.41   -27.95    35.98 2.51        5       12
## betai_ID3          10.90     18.54   -26.74    37.13 2.25        5       12
## betai_ID4          11.41     20.40   -31.00    39.23 2.51        5       12
## betai_ID5          13.79     20.85   -27.07    43.23 2.43        5       12
## betai_ID6          11.51     18.42   -29.35    34.59 2.51        5       12
## betai_ID7          12.06     20.39   -30.37    39.87 2.51        5       12
## betai_ID8          11.90     18.55   -25.80    38.15 2.25        5       12
## betai_ID9          12.15     20.39   -30.28    39.95 2.51        5       12
## betai_ID10         13.01     18.54   -24.65    39.26 2.25        5       12
##
## Further Distributional Parameters:
##          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_mu     1.50      0.42     0.78     2.40 1.00     3087     2630
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
```

```
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
#betai looks bad, but good after unwrapping
#beta0 and betai look bad, but good after unwrapping
UnwrapRhats(model_var,
            variable = '^b_beta')
```

```
## b_beta0_Intercept       b_betai_ID1       b_betai_ID2       b_betai_ID3
##           1.00297           1.00292           1.00223           1.00235
##       b_betai_ID4       b_betai_ID5       b_betai_ID6       b_betai_ID7
##           1.00325           1.00267           1.00321           1.00251
##       b_betai_ID8       b_betai_ID9      b_betai_ID10
##           1.00321           1.00250           1.00189
```

The model successfully accounts for individual differences in both directional bias (β_i) and concentration (α_i). Both hyperparameters (κ_μ and σ_κ) are recovered quite accurately in this case, although their effects on estimates for the individuals with the most extreme biases are also noticeable: in the absence of additional information individuals with very low concentration and large deviations from the population mean (high uncertainty) are estimated to be slightly closer to the population mean than their true values (shrinkage). Despite this uncertainty, the population mean direction is successfully recovered, with a wide posterior distribution for β_0 centred close to the true value.

```
draws_var = as_draws_df(model_var)

par(pty = 's')
par(mar = c(0,0,0,0),
    mfrow = c(3,4))
for(i in 1:length(id_var) ) {
  mu_name = paste0('b_betai_ID',i)
  kappa_name = paste0('r_ID__kappa[',i,',Intercept]')
  PCfun(id_var[[i]],
        col = col_obs,
        sep = 0.05,
        shrink = 1.25,
        plot_rho = FALSE)
  arrows.circular(x = mu_var[i],
                  y = A1(kappa_id_var[i]),
                  col = col_obs,
                  lwd = 5,
                  length = 0.1/1.25 )
  Draws2Cont(draws_var,
             x_string = 'sin(b_beta0_Intercept + get(mu_name)) *A1(softplus(Intercept_kappa+get(kappa_na
             y_string = 'cos(b_beta0_Intercept + get(mu_name))* A1(softplus(Intercept_kappa+get(kappa_na
  with(draws_var,
       arrows.circular(x = median.circular(
         circular(x = mod_circular(b_beta0_Intercept + get(mu_name)),
                  units = 'radians',
                  rotation = 'clock',
                  zero = pi/2) )[1],
         y = A1(softplus(median(Intercept_kappa+get(kappa_name)))),
         lwd = 2, length = 0.1/1.25,
         col = adjustcolor(col_sd, alpha.f = 200/255)) )
  }

#Add the population of biases
DescriptCplot(k = kappa_mu_var,
```

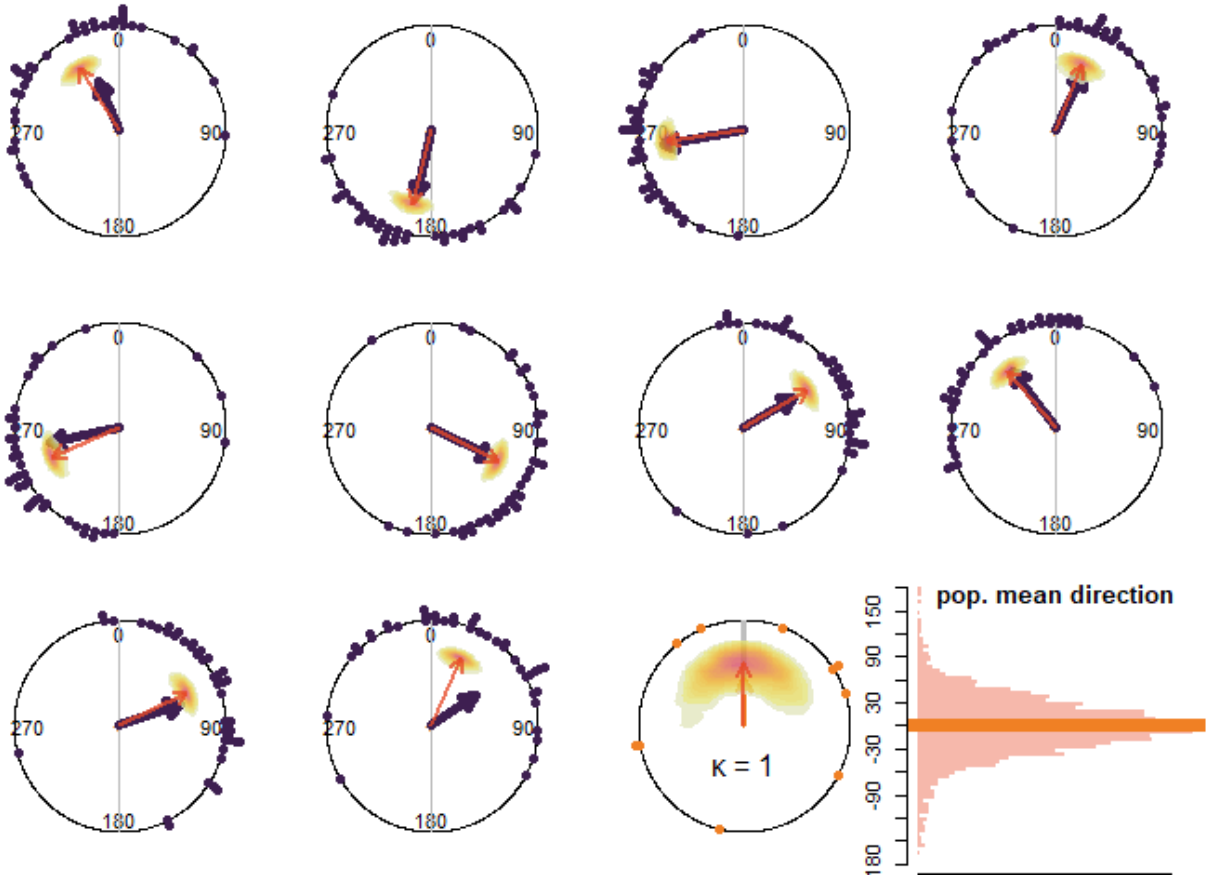
```
              ndata = 10,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = NA,
              mvcol = col_rho )
points.circular(mu_var,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho )
Draws2Cont(draws = draws_var,
           x_string = 'sin(b_beta0_Intercept) *A1(kappa_mu)',
           y_string = 'cos(b_beta0_Intercept)* A1(kappa_mu)' )

with(draws_var,
     arrows.circular(x = mean.circular(
       circular(b_beta0_Intercept,
                units = 'radians',
                rotation = 'clock',
                zero = pi/2) )[1],
       y = A1(median(kappa_mu)),
       lwd = 2,
       length = 0.1/1.25,
       col = adjustcolor(col_sd, alpha.f = 200/255)) )

with(draws_var,
     paste0(mean(unwrap_circular_deg(b_beta0_Intercept) > 0)*100, '%') ) #partial pooling allows accura
```

```
## [1] "47.875%"
```

```
with(draws_var,
     VertHist(data = unwrap_circular_deg(b_beta0_Intercept),
              main = '\npop. mean direction',
              ylim = c(-180, 180),
              col = adjustcolor(col_sd,
                                alpha.f = 100/255),
              axes = FALSE,
              cex.axis = 0.7))
abline(h = 0,
       col = col_rho,
       lwd = 7)
axis(1)
axis(2, at = -6:6*30)
```



Individual heading changes parameters

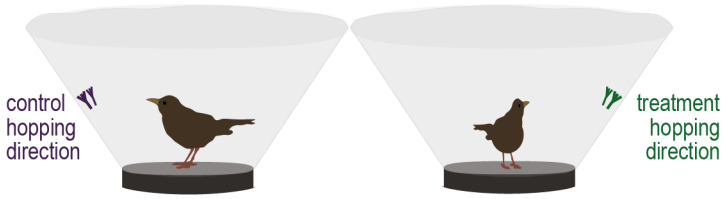


Figure 7: Birds exhibiting migratory restlessness hop repeatedly in their intended flight direction, allowing this to be recorded repeatedly for the same individuals across multiple nights. Individuals may differ somewhat in their hopping directions, but are generally aligned provided they detect the same compass cues. A treatment that affects their perception of these cues causes the entire population to change their hopping direction.

In many experimental designs, we are interested in determining the effect of an experimental treatment on orientation behaviour, just as we did for our examples without repeated-measures. In the context of multiple individuals, the treatment may affect μ and κ for the average individual, as well as the hyperparameters κ_μ and σ_κ that define the distributions of individual differences. As for the earlier examples, we can model the effect of treatment as the change in each parameter value as compared with its value for the control (ideally in the same individual).

In this example, the treatment (m) induces a rightwards turn of 30° (β_m). The average individual's headings are concentrated in both the control ($\alpha_0 = 3.0$) and treatment ($\kappa = \alpha_0 + \alpha_m$, $\alpha_m = 0$), but vary greatly

between individuals ($\sigma_\kappa = 2.5$): in a larger population we might expect 12% of individuals to produce a uniform distribution of headings. This variation is unaffected by the treatment ($\sigma_{\kappa,m} = 0$). Heading biases are highly concentrated across individuals ($\kappa_\mu = 4.0$), which is also unaffected by the treatment ($\kappa_{\mu,m} = 0$).

```
#condition level change in heading of 30°
delta_mu_hd = circular(x = 30,
                    units = 'degrees',
                    rotation = 'clock',
                    zero = pi/2)
#individual differences with kappa = 3
kappa_mu_hd = 4.0
#mean individual highly concentrated
kappa_hd_mean = 3.0
#large variance in individual accuracy
kappa_hd_sd = 2.5
#strong possibilty of uniform distribution
print( paste0( round(
  pnorm(q = 0,
      mean =  kappa_hd_mean,
      sd =  kappa_hd_sd) * 100), "%")
)
```

```
## [1] "12%"
```

```
set.seed(0120810506)#ISBN Batschelet, 1981
kappa_id_hd = rnorm(n = ndata/2,
                    mean = kappa_hd_mean,
                    sd = kappa_hd_sd)
#rectified
kappa_id_hd[kappa_id_hd<0] = 0

# list of circular datasets
set.seed(0120810506)#ISBN Batschelet, 1981
mu_hd = rvonmises(n = ndata/2,
                  mu = c0,
                  kappa = kappa_mu_hd)

mu_delta = rvonmises(n = ndata/2,
                     mu = c0+delta_mu_hd,
                     kappa = kappa_mu_hd)



par(pty = 's')
par(mar = c(0,0,0,0))
par(mfrow = c(3,5))
#before turn
id_hd = mapply(m = mu_hd,
                  k = round(kappa_id_hd,2),
                  FUN = DescriptCplot,
                  save_sample = TRUE,
                  ndata = 20,
                  refline = 0,
                  sdcol = NA,
                  denscol = NA,
```

```
                  pcol = col_obs,
                  mvcol = col_obs,
                  lwd = 5,
                  seed = 0120810506, #ISBN Batschelet, 1981
                  SIMPLIFY = FALSE)
#after turn
id_delta = mapply(m = mu_delta,
                    k = round(kappa_id_hd,2),
                    FUN = DescriptCplot,
                    save_sample = TRUE,
                    pcol = col_treat,
                    mvcol = col_treat,
                    lwd = 5,
                    ndata = 20,
                    refline = 0,
                    sdcol = NA,
                    denscol = NA,
                    seed = 1981,#Publication year Batschelet
                    SIMPLIFY = FALSE)

#Add the population of biases
DescriptCplot(m = delta_mu_hd,
              k = kappa_mu_hd,
              ndata = ndata,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = col_sd,
              mvcol = col_sd
)
points.circular(mu_delta,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho
)

#Add decription of the average individual
DescriptCplot(k = kappa_hd_mean,
              ndata = ndata/2,
              refline = 0,
              sdcol = NA,
              denscol = NA,
              pcol = NA,
              cicol = col_rho,
              mvcol = col_rho
)
kappa_id_hd_ci = kappa_hd_mean +
  kappa_hd_sd *
  qnorm(c(0,1) + c(1,-1)*0.05/2)
#rectify
```

```
kappa_id_hd_ci[kappa_id_hd_ci<0] = 0


arrows(x0 = sin(c0),
       x1 = sin(c0),
       y0 = A1(kappa_id_hd_ci[1]),
       y1 = A1(kappa_id_hd_ci[2]),
       lwd = 7,
       col = adjustcolor(col = col_sd2,
                         alpha.f = 100/255),
       length = 0.05,
       angle = 90,
       code = 3,
       lend = 'butt'
)
mtext(text = paste0('(',paste(signif(kappa_id_hd_ci, 2), collapse = ' '), ')'),
      side = 1,
      line = -1)

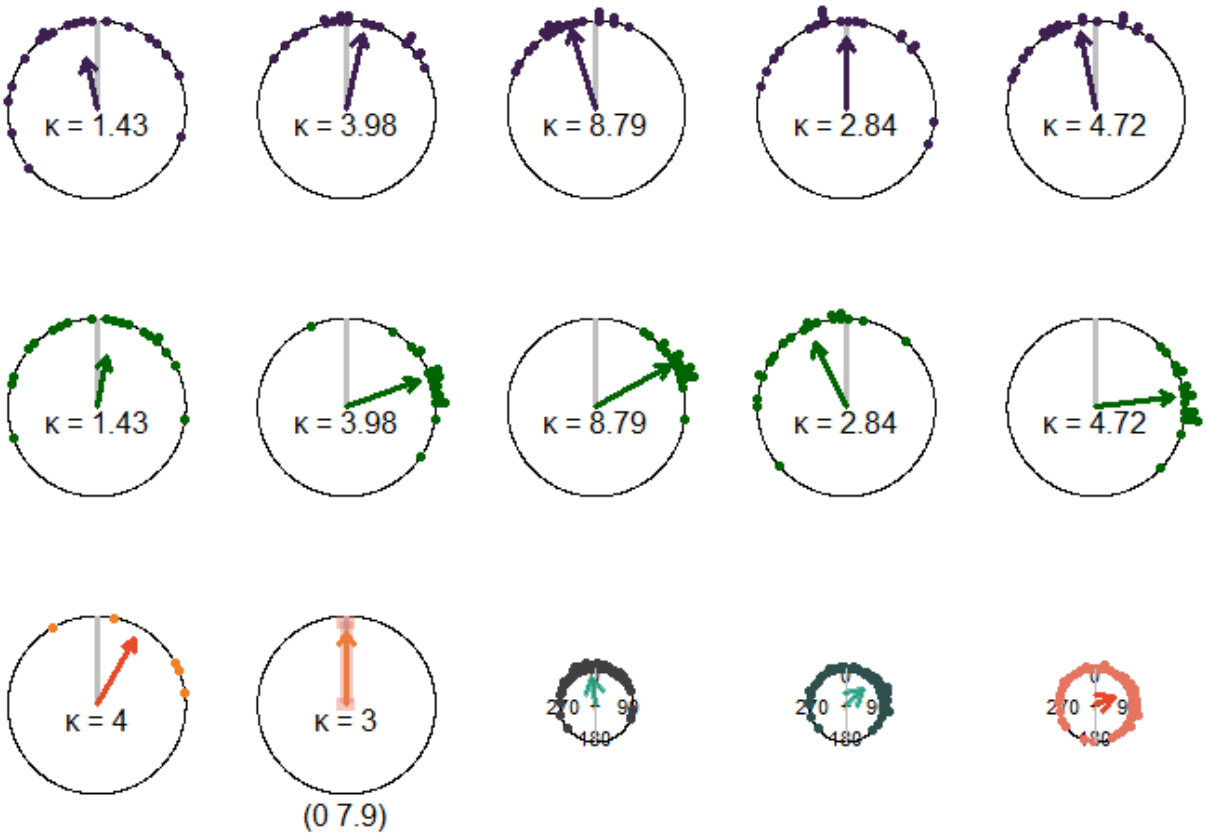

dt_comb_hd = do.call(what = c,
                     args = id_hd)
PCfun(angles = dt_comb_hd,
      col = 'gray25',
      shrink = 3.0)
mle_comb_hd = mle.vonmises(x = dt_comb_hd,bias = TRUE)
ci_comb_hd = with(mle_comb_hd,
                  CI_vM(angles = dt_comb_hd,
                        m1 = mu,
                        k1 = kappa,
                        alternative = 'two.sided')
)
with(mle_comb_hd,
     {
       arrows.circular(x = circular(mu,
                                    units = 'degrees',
                                    rotation = 'clock',
                                    zero = pi/2),
                       y = A1(kappa),
                       lwd = 3,
                       col = col_pdf,
                       length = 0.1
       )
     }
)

dt_comb_delta = do.call(what = c,
                        args = id_delta)
PCfun(angles = dt_comb_delta,
      col = 'darkslategray',
      shrink = 3.0)
mle_comb_delta = mle.vonmises(x = dt_comb_delta,bias = TRUE)
ci_comb_delta = with(mle_comb_delta,
```

```
                    CI_vM(angles = dt_comb_delta,
                          m1 = mu,
                          k1 = kappa,
                          alternative = 'two.sided')
)
with(mle_comb_delta,
     {
       arrows.circular(x = circular(mu,
                                    units = 'degrees',
                                    rotation = 'clock',
                                    zero = pi/2),
                       y = A1(kappa),
                       lwd = 3,
                       col = col_pdf,
                       length = 0.1
       )
     }
)


dt_comb_diffs = dt_comb_delta - dt_comb_hd

PCfun(angles = dt_comb_diffs,
      col = col_sd2,
      shrink = 3.0)
mle_comb_diffs = mle.vonmises(x = dt_comb_diffs,bias = TRUE)
ci_comb_diffs = with(mle_comb_diffs,
                    CI_vM(angles = dt_comb_diffs,
                          m1 = mu,
                          k1 = kappa,
                          alternative = 'two.sided')
)
with(mle_comb_diffs,
     {
       arrows.circular(x = circular(mu,
                                    units = 'degrees',
                                    rotation = 'clock',
                                    zero = pi/2),
                       y = A1(kappa),
                       lwd = 3,
                       col = col_sd,
                       length = 0.1
       )
     }
)
```



Although this group of individuals is likely too small to estimate many of the effects of interest, there is sufficient inter-individual correlation to observe the effect of the treatment: a population-level rightward turn. Nonetheless there are large differences between individual responses: one individual even favours a left turn after treatment. Individuals also differ greatly in the concentration of their headings, ranging from $\kappa = 1.4$ to $\kappa = 8.8$, a range in expected (circular) standard deviation between 60° and 20°.

Fit the model

For this model, we need to update the formula to account for effects of treatment on μ, κ and their respective hyperparameters. We do this by including the treatment vector x as a predictor for β_0 (`beta0 ~ 1 + x`) and α_0 (`kappa ~ 0 + Intercept + x`, where `0 + Intercept` prevents `BRMS` from centring x, so that the intercept represents the control condition rather than the average across conditions), and as an individual-specific predictor of β_i (`betai ~ 0 + ID + x:ID`) and α_i (`((1+ x|ID)`).

```
#collect data
dt_hd = data.frame(y = as.numeric( rad(c(dt_comb_hd, dt_comb_delta)) ),#angles as a numeric vector in r
                   x = c(rep(0, times = length(dt_comb_hd)),
                         rep(1, times = length(dt_comb_delta)) ),
                   ID = factor(x = #individual labels as an unordered categorical (factor)
                                 c(sort(rep(1:length(mu_hd), 20)),
                                   sort(rep(1:length(mu_delta), 20))),
                               ordered = FALSE)
                   )
#set up formula with both 'nonlinear' and linear predictors
form_hd = bf(y ~ mu,
                  nlf(mu ~ beta0 + betai),#a nonlinear formula
```

```
                  beta0 ~ 1 + x,
                  betai ~ 0 + ID + x:ID,
                  kappa ~ 0 + Intercept + x +  (1 + x|ID),#a linear formula
                  family = unwrap_von_mises)
```

While the linear formula for κ automatically generates the corresponding hyperparameter for the effect of treatment on individual differences in concentration (σ_{κ_m}), for the non-linear formula we need to add a treatment effect on the concentration of individual heading biases. This is modelled as a change in κ_μ ($\kappa_{\mu,m}$) the inverse-softplus scale ($\kappa_{\mu,m} \sim \log(\exp(k_{\mu,0} + k_{\mu,x}) + 1)$). This informs the prior for individual heading biases in the treatment condition ($\beta_{i,m} \sim \mathcal{M}(0°, \kappa_{\mu,m})$) in the same way that κ_μ does for β_i in the control. `BRMS` generates variables in Stan that represent the number of individuals (`N_1`) and number of conditions (`M_1`). Since there are only two conditions, and all individuals experience both treatments once, we can identify the individual parameters for the control as 'slices' of the `b_betai` vector: `b_betai[1:N_1]` and those for the treatment as `b_betai[(N_1+1):(M_1*N_1)]`. By default we expect no effect of treatment on κ_μ, but moderate increases or decreases are quite plausible, so an appropriate prior is a normal distribution centred at no-change: $\kappa_{\mu,m} \sim \mathcal{N}(0, 1)$.

```
#the individual effects on change in heading need a new parameter
stan_kappamux = stanvar(scode = "
real softkappamux;
                           ",
                        block = "parameters") +
  stanvar(scode = "
real kappa_mu_x = log1p_exp(softkappamu + softkappamux);
          ",
          block = 'genquant')

prior_hd = prior('normal(0, 3*pi()/2)',class = 'b', nlpar = 'beta0') + #wider prior helps avoid bias
  prior('normal(0, pi()/2)',class = 'b', nlpar = 'beta0', coef = 'x') + #expectation of moderate sized
  set_prior(prior = 'target += unwrap_von_mises_vect_lpdf(b_betai[1:N_1] | 0, log1p_exp(softkappamu))',
set_prior(prior = 'target += unwrap_von_mises_vect_lpdf(b_betai[1:N_1] + b_betai[(N_1+1):(M_1*N_1)] | 0
set_prior(prior = 'target += lognormal_lpdf(log1p_exp(softkappamu) | log(15), 0.6)', check = FALSE) + #
set_prior(prior = 'target += normal_lpdf(softkappamux | 0, 1.0)', check = FALSE) + #expect only small e
  prior('normal( 3.0, 3.0)', class = 'b', dpar = 'kappa', coef = 'Intercept') + #shouldn't be too tight
  prior('normal( 0.0, 3.0)', class = 'b', dpar = 'kappa', coef = 'x') + #turns should not affect popula
  prior('student_t(3, 0, 3.0)', class = 'sd', dpar = 'kappa') +  #now expect substantial variation, but
  prior('student_t(3, 0, 3.0)', class = 'sd', dpar = 'kappa', coef = 'x', group = 'ID') #now expect sub

#run the model
model_hd = brm(
           formula = form_hd,
           data = dt_hd,
           family = unwrap_von_mises,
           stanvars = stan_unwrap_fun +
                      modulo_circular_fun +
                      stan_softkappamu +
                      stan_kappamux,
           prior = prior_hd,
           silent = 2,
           cores = 4,
           backend = 'cmdstan')
```

```
sm_hd = summary(model_hd)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
```

```
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
print(sm_hd, digits = 2)
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 1 + x
##          betai ~ 0 + ID + x:ID
##          kappa ~ 0 + Intercept + x + (1 + x | ID)
##    Data: dt_hd (Number of observations: 200)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 5)
##                               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
## sd(kappa_Intercept)               2.48      1.20     0.85     5.38 1.00     2704
## sd(kappa_x)                       2.14      1.54     0.12     5.82 1.00     1776
## cor(kappa_Intercept,kappa_x)      0.49      0.45    -0.63     0.99 1.00     3201
##                               Tail_ESS
## sd(kappa_Intercept)               2744
## sd(kappa_x)                       1473
## cor(kappa_Intercept,kappa_x)      2669
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## beta0_Intercept    -0.15      0.21    -0.58     0.27 1.00     1270     1588
## beta0_x             0.85      0.33     0.19     1.51 1.00     1285     2099
## betai_ID1          -1.67      2.74    -6.72     0.40 1.53        7       35
## betai_ID2          -1.33      2.74    -6.35     0.68 1.53        7       31
## betai_ID3          -0.21      0.23    -0.66     0.24 1.00     1380     2111
## betai_ID4           0.10      0.25    -0.40     0.59 1.00     1637     2376
## betai_ID5          -3.26      3.14    -6.75     0.27 1.74        6      135
## betai_ID1:x        -0.46      4.47    -7.25     6.33 2.41        5       28
## betai_ID2:x         0.25      0.35    -0.41     0.97 1.00     1462     2176
## betai_ID3:x        -1.00      2.74    -6.15     1.21 1.53        7       29
## betai_ID4:x        -1.28      0.39    -2.06    -0.55 1.00     1658     2288
## betai_ID5:x         0.87      0.35     0.19     1.56 1.00     1477     2371
## kappa_Intercept     3.19      1.16     0.95     5.62 1.00     1878     2095
## kappa_x             1.80      1.27    -0.41     4.54 1.00     2549     2318
##
## Further Distributional Parameters:
##            Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_mu       5.66      1.81     2.80     9.95 1.00     3572     2298
## kappa_mu_x     4.95      1.87     1.98     9.22 1.00     3467     2345
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
#betai looks bad, but good after unwrapping
#beta0 and betai look bad, but good after unwrapping
```

```
UnwrapRhats(model_hd,
            variable = '^b_beta')
```

```
## b_beta0_Intercept         b_beta0_x       b_betai_ID1       b_betai_ID2
##           1.00149           1.00110           1.00136           1.00241
##       b_betai_ID3       b_betai_ID4       b_betai_ID5     b_betai_ID1:x
##           1.00158           1.00398           1.00102           1.00176
##     b_betai_ID2:x     b_betai_ID3:x     b_betai_ID4:x     b_betai_ID5:x
##           1.00221           1.00075           1.00185           1.00076
```

From the raw coefficients, we can already see that parameter recovery has been quite successful for κ and κ_μ, given the minimal sample size. Both the population mean κ and the (inverse-softplus scaled) estimates of σ_κ are close to their true values. The data for the treatment are quite different from those in the control, even though their true generative distribution differs only in population mean direction ($\mu = \beta_0 + \beta_m$). As a result there is some uncertainty regarding the effect of treatment x on κ, κ_μ and σ_κ. For `kappa_x` there is substantial posterior density for both increases and decreases in concentration in the treatment condition (95% CI: -0.41–4.54), and the range of estimates for `kappa_mu_x` ($\log(\exp(k_{\mu,0} + k_{\mu,x}) + 1)$) almost entirely overlaps with that for `kappa_mu` in the control condition. Large inter-individual variation in treatment effects on concentration (`sd(kappa_x)`) are estimated, and their correlation with individual differences in concentration for the control also has substantial posterior density for both correlation and anti-correlation (95% CI `cor(kappa_Intercept,kappa_x)`: -0.63–0.99), so there is no clear evidence of a relationship between variation in the two conditions (correctly, since none was simulated).

Test hypothesis of treatment effect

Despite the uncertainty in this dataset, it should still be possible to test the hypothesis of a population-level change in direction. We can do this in the same way as for the example without repeated measures: by fitting a model in which x affects κ ('kappa only'), but not μ, and one in which there is no effect of x ('null model'). Since differences between individuals could still be present if either hypothesis is true, all individual effects (β_i & α_i) and hyperparameters (κ_μ and σ_κ) unrelated to the treatment effect should remain in all models. Due to the small sample size, individual observations can greatly affect model fit. To achieve robust model comparison under these circumstances, we will need to fit all models with the settings `save_pars(all = TRUE)`, which allows for moment matching computation of exact cross-validation for problematic observations.

```
form_hd_kappa = bf(y ~ mu,
                   nlf(mu ~ beta0 + betai),#a nonlinear formula
                   beta0 ~ 1,
                   betai ~ 0 + ID,
                   kappa ~ 0 + Intercept + x +  (1 + x|ID),#a linear formula
                   family = unwrap_von_mises)

form_hd_null = bf(y ~ mu,
                   nlf(mu ~ beta0 + betai),#a nonlinear formula
                   beta0 ~ 1,
                   betai ~ 0 + ID,
                   kappa ~ 1 + (1 |ID),#a linear formula
                   family = unwrap_von_mises)

#without effects of x on mu
prior_hd_kappa = prior('normal(0, 3*pi()/2)',class = 'b', nlpar = 'beta0') + #wider prior helps avoid b
  set_prior(prior = 'target += unwrap_von_mises_vect_lpdf(b_betai | 0, log1p_exp(softkappamu))', check =
set_prior(prior = 'target += lognormal_lpdf(log1p_exp(softkappamu) | log(15), 0.6)', check = FALSE) + #
  prior('normal( 3.0, 3.0)', class = 'b', dpar = 'kappa', coef = 'Intercept') + #shouldn't be too tight
  prior('normal( 0.0, 3.0)', class = 'b', dpar = 'kappa', coef = 'x') + #turns should not affect popula
```

```
  prior('student_t(3, 0, 3.0)', class = 'sd', dpar = 'kappa') +  #now expect substantial variation, but
  prior('student_t(3, 0, 3.0)', class = 'sd', dpar = 'kappa', coef = 'x', group = 'ID') #now expect sub

#without effects of x
prior_hd_null = prior('normal(0, 3*pi()/2)',class = 'b', nlpar = 'beta0') + #wider prior helps avoid bi
  set_prior(prior = 'target += unwrap_von_mises_vect_lpdf(b_betai | 0, log1p_exp(softkappamu))', check =
set_prior(prior = 'target += lognormal_lpdf(log1p_exp(softkappamu) | log(15), 0.6)', check = FALSE) + #
  prior('normal( 3.0, 3.0)', class = 'Intercept', dpar = 'kappa') + #shouldn't be too tight, want to es
  prior('student_t(3, 0, 3.0)', class = 'sd', dpar = 'kappa')  #now expect substantial variation, but t


#refit and save all parameters for model comparison
model_hd_treatment = brm(
  formula = form_hd,
  data = dt_hd,
  family = unwrap_von_mises,
  stanvars = stan_unwrap_fun +
                     modulo_circular_fun +
                     stan_softkappamu +
                     stan_kappamux,
  prior = prior_hd,
  cores = 4,
  silent = 2, # without printing
  backend = 'cmdstan',
  save_pars = save_pars(all = TRUE),
  control = list(adapt_delta = 0.97)#slower, but more robust sampling
)

model_hd_kappa = brm(
  formula = form_hd_kappa,
  data = dt_hd,
  family = unwrap_von_mises,
  stanvars = stan_unwrap_fun +
                     modulo_circular_fun +
                     stan_softkappamu, #without the kappamux variable
  prior = prior_hd_kappa,
  cores = 4,
  silent = 2, # without printing
  save_pars = save_pars(all = TRUE),
  backend = 'cmdstan',
  control = list(adapt_delta = 0.97)#slower, but more robust sampling
)

model_hd_null = brm(
  formula = form_hd_null,
  data = dt_hd,
  family = unwrap_von_mises,
  stanvars = stan_unwrap_fun +
                     modulo_circular_fun +
                     stan_softkappamu,#without the kappamux variable
  prior = prior_hd_null,
  cores = 4,
  silent = 2, # without printing
```

```
  save_pars = save_pars(all = TRUE),
  backend = 'cmdstan',
  control = list(adapt_delta = 0.97)#slower, but more robust sampling
)
```

```
summary(model_hd_treatment)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 1 + x
##          betai ~ 0 + ID + x:ID
##          kappa ~ 0 + Intercept + x + (1 + x | ID)
##    Data: dt_hd (Number of observations: 200)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 5)
##                               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
## sd(kappa_Intercept)               2.44      1.18     0.87     5.44 1.00     1830
## sd(kappa_x)                       2.16      1.57     0.14     5.95 1.00     1668
## cor(kappa_Intercept,kappa_x)      0.51      0.44    -0.59     0.99 1.00     2416
##                               Tail_ESS
## sd(kappa_Intercept)               2619
## sd(kappa_x)                       1448
## cor(kappa_Intercept,kappa_x)      2241
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## beta0_Intercept    -0.15      0.21    -0.54     0.28 1.00      798     1327
## beta0_x             0.86      0.32     0.21     1.48 1.00      920     1354
## betai_ID1          -0.10      0.27    -0.63     0.43 1.00     1433     1751
## betai_ID2           0.24      0.23    -0.21     0.70 1.00      961     1599
## betai_ID3          -0.21      0.22    -0.66     0.21 1.00      872     1415
## betai_ID4           0.10      0.25    -0.39     0.59 1.00     1166     1721
## betai_ID5          -0.12      0.22    -0.58     0.31 1.00      926     1559
## betai_ID1:x         1.11      2.74    -1.17     6.32 1.53        7       30
## betai_ID2:x         0.25      0.35    -0.43     0.96 1.00     1055     1414
## betai_ID3:x         0.57      0.34    -0.08     1.24 1.00      977     1482
## betai_ID4:x        -2.86      2.74    -8.06    -0.62 1.53        7       30
## betai_ID5:x         0.86      0.34     0.22     1.53 1.00     1032     1447
## kappa_Intercept     3.22      1.15     0.98     5.67 1.00     1500     1899
## kappa_x             1.77      1.25    -0.56     4.47 1.00     1944     2013
##
## Further Distributional Parameters:
##            Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_mu       5.75      1.93     2.82    10.30 1.00     2570     1592
## kappa_mu_x     5.04      2.03     1.95     9.86 1.00     2806     1801
```

```
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
UnwrapRhats(model_hd_treatment,
            variable = '^b_beta')
```

```
## b_beta0_Intercept         b_beta0_x       b_betai_ID1       b_betai_ID2
##           1.00039           1.00155           1.00065           1.00008
##       b_betai_ID3       b_betai_ID4       b_betai_ID5     b_betai_ID1:x
##           1.00015           1.00007           1.00032           1.00172
##     b_betai_ID2:x     b_betai_ID3:x     b_betai_ID4:x     b_betai_ID5:x
##           1.00109           1.00099           1.00039           1.00113
```

```
summary(model_hd_kappa)
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 1
##          betai ~ 0 + ID
##          kappa ~ 0 + Intercept + x + (1 + x | ID)
##    Data: dt_hd (Number of observations: 200)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 5)
##                               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
## sd(kappa_Intercept)               1.89      1.02     0.62     4.41 1.00     1675
## sd(kappa_x)                       5.69      2.23     2.46    11.12 1.00     1871
## cor(kappa_Intercept,kappa_x)     -0.75      0.28    -0.99     0.02 1.00     1543
##                               Tail_ESS
## sd(kappa_Intercept)               2087
## sd(kappa_x)                       2278
## cor(kappa_Intercept,kappa_x)      2349
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## beta0_Intercept     0.63      0.25     0.13     1.12 1.00     1276     1574
## betai_ID1          -0.64      0.28    -1.19    -0.12 1.00     1459     1946
## betai_ID2           0.32      0.28    -0.24     0.86 1.00     1590     2078
## betai_ID3           0.41      0.26    -0.10     0.93 1.00     1359     1607
## betai_ID4          -0.93      0.28    -1.50    -0.41 1.00     1518     2184
## betai_ID5           0.80      0.26     0.29     1.32 1.00     1243     1742
## kappa_Intercept     0.72      0.84    -0.83     2.51 1.00     1377     1455
## kappa_x             2.91      2.07    -1.67     6.55 1.01     1805     1809
##
## Further Distributional Parameters:
##          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_mu     5.10      1.86     2.25     9.29 1.00     3215     2225
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
```

```
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
UnwrapRhats(model_hd_kappa,
            variable = '^b_beta')
```

```
## b_beta0_Intercept       b_betai_ID1       b_betai_ID2       b_betai_ID3
##           1.00074           1.00017           0.99997           1.00057
##       b_betai_ID4       b_betai_ID5
##           1.00081           1.00057
```

```
summary(model_hd_null)
```

```
## Warning: Parts of the model have not converged (some Rhats are > 1.05). Be
## careful when analysing the results! We recommend running more iterations and/or
## setting stronger priors.
```

```
##  Family: unwrap_von_mises
##   Links: mu = identity; kappa = softplus
## Formula: y ~ mu
##          mu ~ beta0 + betai
##          beta0 ~ 1
##          betai ~ 0 + ID
##          kappa ~ 1 + (1 | ID)
##    Data: dt_hd (Number of observations: 200)
##   Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##          total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~ID (Number of levels: 5)
##                     Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(kappa_Intercept)     0.43      0.41     0.01     1.45 1.00     1255     1655
##
## Regression Coefficients:
##                 Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_Intercept     1.65      0.30     1.06     2.28 1.00     2032     1909
## beta0_Intercept     0.26      0.17    -0.08     0.58 1.00     1184     1628
## betai_ID1          -0.30      0.20    -0.71     0.10 1.00     1742     2116
## betai_ID2          -1.21      2.73    -6.18     0.73 1.53        7       35
## betai_ID3           0.10      0.19    -0.27     0.48 1.00     1449     1925
## betai_ID4          -0.49      0.19    -0.89    -0.13 1.00     1466     2214
## betai_ID5           0.31      0.20    -0.07     0.73 1.00     1779     1828
##
## Further Distributional Parameters:
##          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## kappa_mu    10.94      5.22     3.88    24.63 1.00     1722     1078
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
UnwrapRhats(model_hd_null,
            variable = '^b_beta')
```

```
## b_beta0_Intercept       b_betai_ID1       b_betai_ID2       b_betai_ID3
##           1.00131           1.00230           1.00189           1.00138
##       b_betai_ID4       b_betai_ID5
```

```
##           1.00206           0.99990
```

To compare the predictive power of the models, we can use leave-one-out cross-validation (LOO-CV) to calculate the estimated log predictive density (ELPD) and its standard error for each model. The more robustly the model predicts the data, the higher the total ELPD. In this case a moment-matching algorithm is used to compensate for observations that have a strong effect on the model posterior (high Pareto-$\hat{k}$ value). This takes longer, since models need to be recompiled with `rstan`, but improves the accuracy of model comparison.

```
#better predictions should justify fitting a treatment model
loo_null = loo(model_hd_null, moment_match = TRUE)
```

```
## Recompiling the model with 'rstan'
```

```
## Recompilation done
```

```
loo_kappa = loo(model_hd_kappa, moment_match = TRUE, reloo = TRUE)# Pareto-k warning
```

```
## Recompiling the model with 'rstan'
## Recompilation done
```

```
## No problematic observations found. Returning the original 'loo' object.
```

```
loo_treat = loo(model_hd_treatment, moment_match = TRUE)
```

```
## Recompiling the model with 'rstan'
```

```
## Recompilation done
```

```
lc_nulltreat = loo_compare(loo_null, loo_kappa, loo_treat)
print(lc_nulltreat)
```

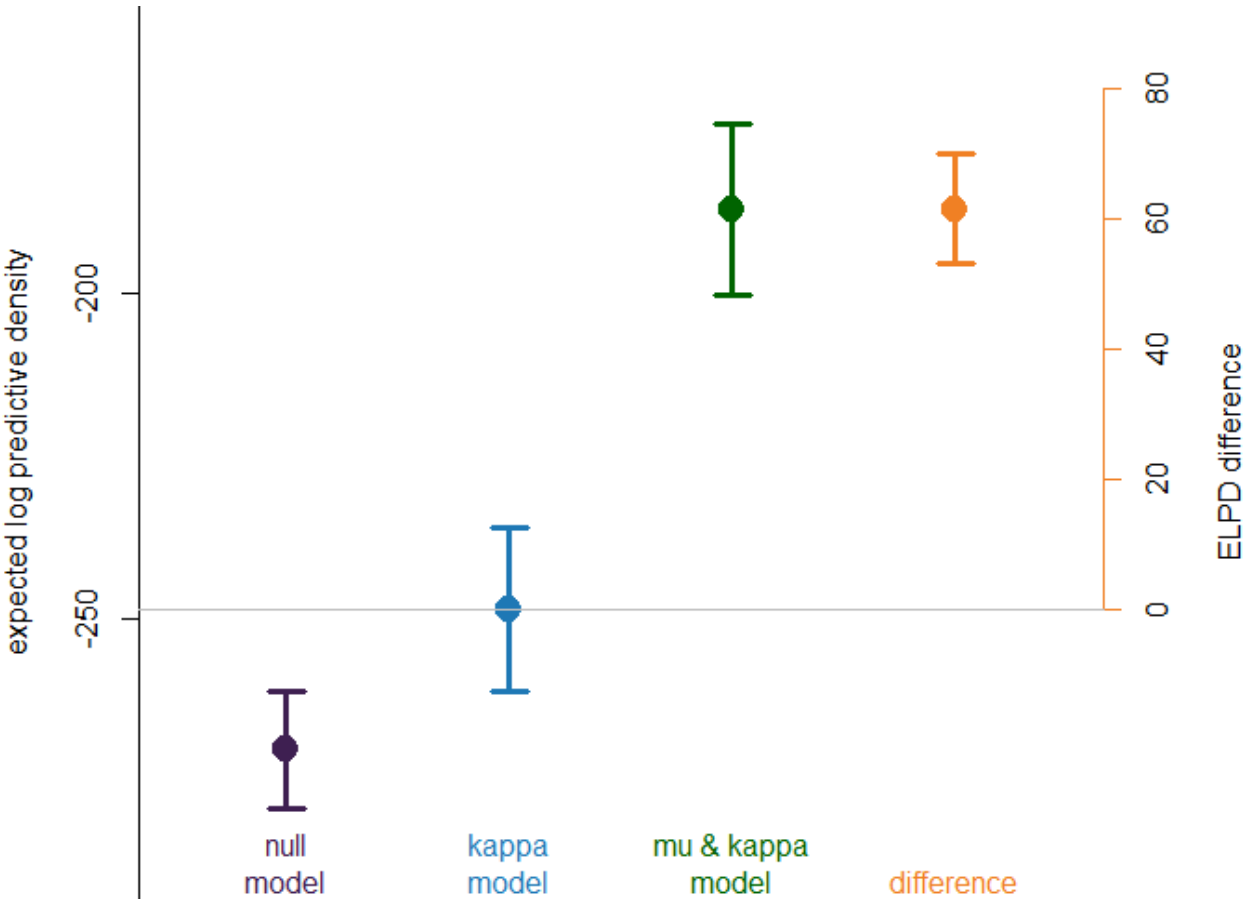
```
##                      elpd_diff se_diff
## model_hd_treatment     0.0       0.0
## model_hd_kappa       -61.5       8.4
## model_hd_null        -83.0      11.1
```

In this case, the results of model comparison are quite clear, the model for the hypothesis of an effect of treatment on both μ and κ predicts the data more robustly than the other two hypotheses (ELPD difference from the κ model: 61.5 ± 8.4; from the null model: 83.0 ± 11.1). We can plot this model comparison to get an impression of the scale of the difference in plausibility between the hypotheses.

```
lc_plot = data.frame(elpd = c(loo_null$estimates['elpd_loo','Estimate'],
                              loo_kappa$estimates['elpd_loo','Estimate'],
                              loo_treat$estimates['elpd_loo','Estimate'],
                              loo_kappa$estimates['elpd_loo','Estimate'] - lc_nulltreat[2,'elpd_diff'])
                     se = c(loo_null$estimates['elpd_loo','SE'],
                            loo_kappa$estimates['elpd_loo','SE'],
                            loo_treat$estimates['elpd_loo','SE'],
                            lc_nulltreat[2,'se_diff'])
                     )

par(mar = c(0,4,0,4),
    mfrow = c(1,1))
plot(x = 1:dim(lc_plot)[1],
     y = lc_plot$elpd,
     xlab = '',
     ylab = 'expected log predictive density',
     xlim = c(1,dim(lc_plot)[1]) + c(-1,1)*0.5,
     ylim = with(lc_plot, {range(elpd+se%*% t(c(-2,2)))}), #within 2sigma of all estimates
```

```
     pch = 19,
     col = c(col_obs, col_kappa, col_treat, col_rho),
     cex = 2,
     axes = FALSE)
with(lc_plot,
     {
arrows(x0 = 1:dim(lc_plot)[1],
       x1 = 1:dim(lc_plot)[1],
       y0 = elpd - se,
       y1 = elpd + se,
       code = 3,
       angle = 90,
       length = 0.1,
       lwd = 3,
       col =  c(col_obs, col_kappa, col_treat, col_rho)
       )
     }
)
axis(2,
     at = pretty(c(0,
                   with(lc_plot, {range(elpd+se%*% t(c(-2,2)))}))
                 )
     )
axis(4,
     at = with(lc_plot,
               {
               seq(from = elpd[2], to = elpd[dim(lc_plot)[1]] + se[dim(lc_plot)[1]]*4, by = 20)
               }
               ),
     labels = with(lc_plot,
                           {
                             seq(from = 0, to =  elpd[dim(lc_plot)[1]] + se[dim(lc_plot)[1]]*4 - elpd[2]
                           }
     ),
     col = col_rho
)
abline(h = lc_plot$elpd[2],
       col = 'gray')
mtext(text = 'ELPD difference',
      side = 4,
      line = 3
      )
mtext(side = 1,
      line = -1,
      at = 1:dim(lc_plot)[1],
     text = c('null\nmodel',
              'kappa\nmodel',
              'mu & kappa\nmodel',
              'difference'),
     col = c(col_obs, col_kappa, col_treat, col_rho)
     )
```



Plot model predictions

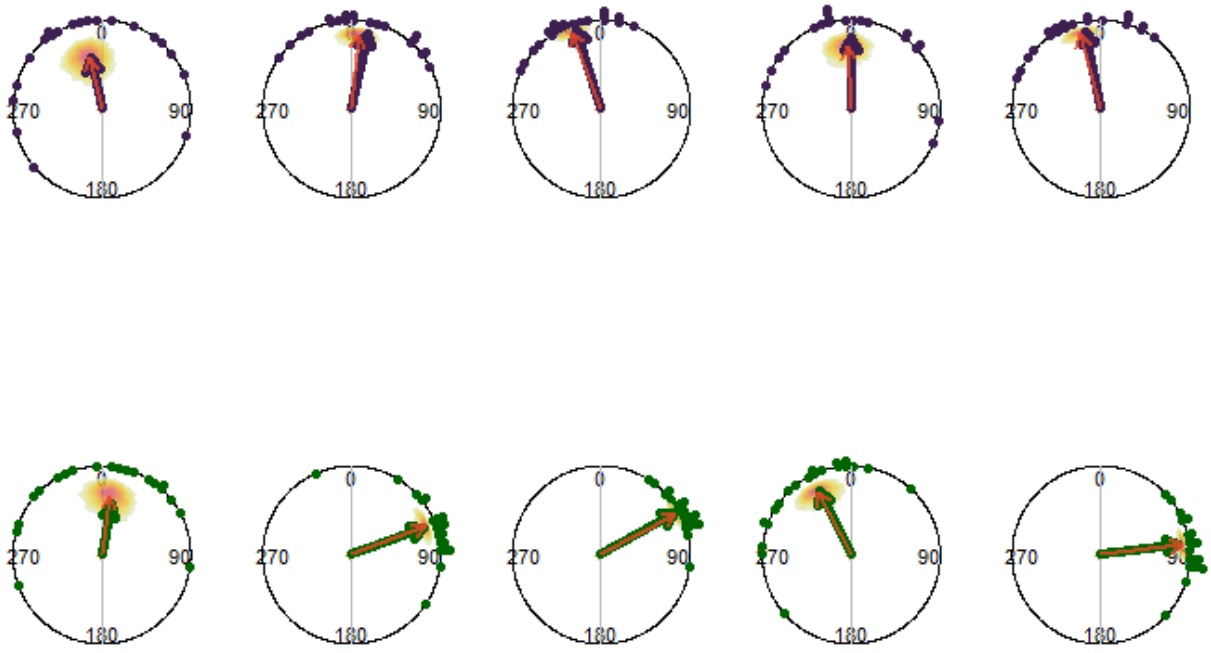
Now that we have confirmed that this model is the most appropriate (from our chosen set of hypotheses) we extract the model predictions. Since the data for the control have different predictors from those in the treatment, that may differ between individuals, these predictions need to be calculated separately for each condition for each individual.

```
draws_treat = as_draws_df(model_hd_treatment)

par(pty = 's')
par(mar = c(0,0,0,0),
    mfrow = c(2,length(mu_hd)))
#control
for(i in 1:length(mu_hd) )
{
  mu_name = paste0('b_betai_ID',i)
  kappa_name = paste0('r_ID__kappa[',i,',Intercept]')
  PCfun(id_hd[[i]],
        col = col_obs,
        sep = 0.05,
        shrink = 1.25,
        plot_rho = FALSE)
  Draws2Cont(draws_treat,
             x_string = 'sin(b_beta0_Intercept + get(mu_name))*
                         A1(softplus(b_kappa_Intercept+get(kappa_name)))',
```

```
            y_string = 'cos(b_beta0_Intercept + get(mu_name))*
                        A1(softplus(b_kappa_Intercept+get(kappa_name)))',
            ngrid = 100, # needs higher resolution to avoid spillover
            cropc = TRUE
  )
  arrows.circular(x = mu_hd[i],
                  y = A1(kappa_id_hd[i]),
                  col = col_obs,
                  lwd = 5,
                  length = 0.1/1.25
  )
  with(draws_treat,
       arrows.circular(x = median.circular(
         circular(x =
                    mod_circular(b_beta0_Intercept + get(mu_name)),
                  units = 'radians',
                  rotation = 'clock',
                  zero = pi/2)
       )[1],
       y = A1(softplus(median(b_kappa_Intercept+get(kappa_name)))),
       lwd = 2,
       length = 0.1/1.25,
       col = adjustcolor(col_sd, alpha.f = 200/255))
  )
}
#treatment
for(i in 1:length(mu_delta) )
{
  mu_name = paste0('b_betai_ID',i)
  mu_delta_name = paste0('b_betai_ID',i , ':x')
  kappa_name = paste0('r_ID__kappa[',i,',Intercept]')
  kappa_delta_name = paste0('r_ID__kappa[',i,',x]')
  PCfun(id_delta[[i]],
        col = col_treat,
        sep = 0.05,
        shrink = 1.25,
        plot_rho = FALSE)
  Draws2Cont(draws_treat,
            x_string = 'sin(b_beta0_Intercept + b_beta0_x + get(mu_name) + get(mu_delta_name))*
                        A1(softplus(b_kappa_Intercept+b_kappa_x+get(kappa_name)+get(kappa_delta_name)))
            y_string = 'cos(b_beta0_Intercept + b_beta0_x + get(mu_name) + get(mu_delta_name))*
                        A1(softplus(b_kappa_Intercept+b_kappa_x+get(kappa_name)+get(kappa_delta_name)))
            ngrid = 100, # needs higher resolution to avoid spillover
            cropc = TRUE
           )
  arrows.circular(x = mu_delta[i],
                  y = A1(kappa_id_hd[i]),
                  col = col_treat,
                  lwd = 5,
                  length = 0.1/1.25
  )
  with(draws_treat,
       arrows.circular(x = median.circular(
```

```
          circular(x =
                     mod_circular(b_beta0_Intercept+ b_beta0_x + get(mu_name)  + get(mu_delta_name)),
                   units = 'radians',
                   rotation = 'clock',
                   zero = pi/2)
        )[1],
        y = A1(softplus(median(b_kappa_Intercept + b_kappa_x + get(kappa_name) + get(kappa_delta_name)))),
        lwd = 2,
        length = 0.1/1.25,
        col = adjustcolor(col_sd, alpha.f = 200/255))
  )
}
```



We can also compare the model parameters with the population parameters. Here we see that the estimates of the population mean angle favour a right turn in 99.4% of estimates, but there is a lot of uncertainty about the exact size of the turn (95% CI: 12° to 85°; true turn 30°). This is influenced by the wide spread of individual mean angles and turn angles, which the model recovered well.

```
par(mfrow = c(1,4))
#Add the population of biases
DescriptCplot(k = kappa_mu_hd,
              ndata = length(mu_hd),
              refline = 0,
              denscol = NA,
              pcol = NA,
              sdcol = NA,
```

```
              mvcol = col_obs
)
points.circular(mu_hd,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho
)

Draws2Cont(draws = draws_treat,
           x_string = 'sin(b_beta0_Intercept)*
             A1(kappa_mu)',
           y_string = 'cos(b_beta0_Intercept)*
             A1(kappa_mu)',
           ngrid = 100, # needs higher resolution to avoid spillover
           cropc = TRUE
)

with(draws_treat,
     arrows.circular(x = mean.circular(circular(b_beta0_Intercept,
                                                units = 'radians',
                                                rotation = 'clock',
                                                zero = pi/2)
     )[1],
     y = A1(median(kappa_mu)),
     lwd = 2,
     length = 0.1/1.25,
     col = adjustcolor(col_sd, alpha.f = 200/255))
)

#Add the population of turns
DescriptCplot(m = delta_mu_hd,
              k = kappa_mu_hd,
              ndata = length(mu_hd),
              refline = 0,
              denscol = NA,
              pcol = NA,
              sdcol = NA,
              mvcol = col_treat
)
points.circular(mu_delta,
                bins = 360/5-1,
                stack = TRUE,
                sep = 0.05,
                shrink = 1.25,
                col = col_rho
)

Draws2Cont(draws = draws_treat,
           x_string = 'sin(b_beta0_x)*
             A1(kappa_mu_x)',
           y_string = 'cos(b_beta0_x)*
```

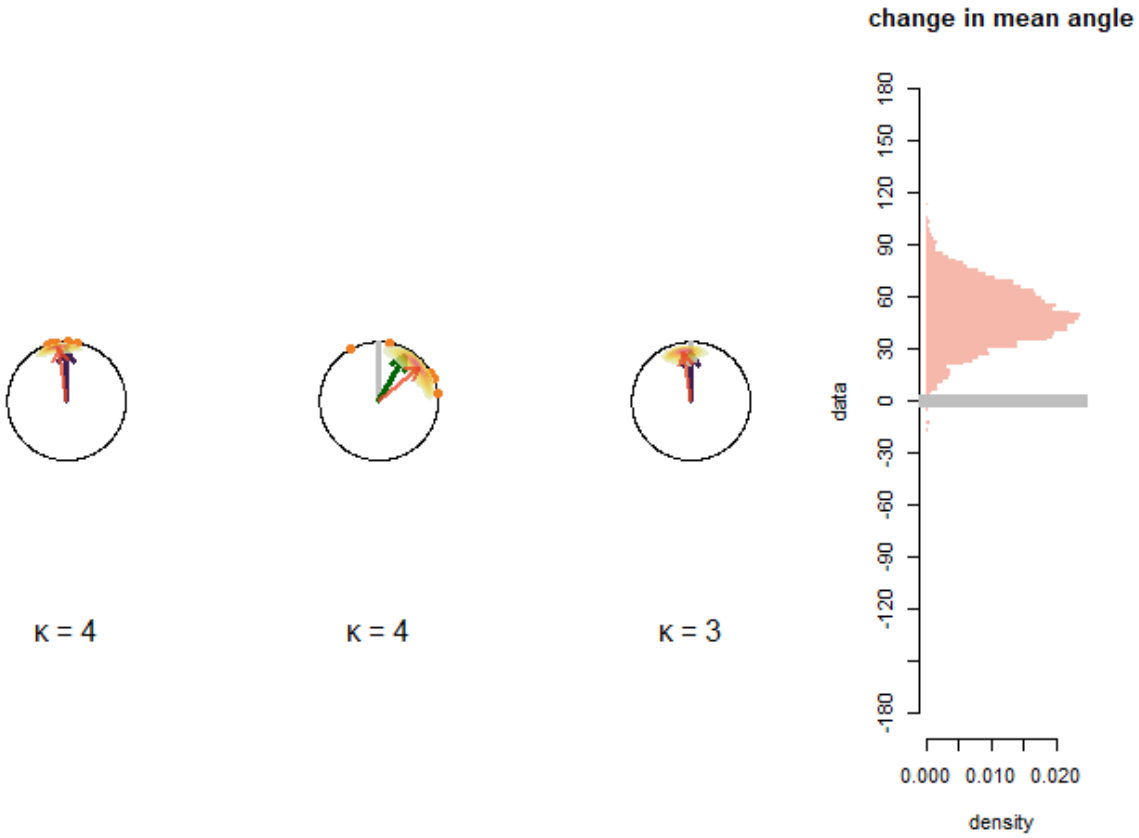
```
            A1(kappa_mu_x)',
          ngrid = 100, # needs higher resolution to avoid spillover
          cropc = TRUE
)
with(draws_treat,
     arrows.circular(x = mean.circular(circular(b_beta0_x,
                                                units = 'radians',
                                                rotation = 'clock',
                                                zero = pi/2)
     )[1],
     y = A1(median(kappa_mu_x)),
     lwd = 2,
     length = 0.1/1.25,
     col = adjustcolor(col_sd, alpha.f = 200/255))
)

#Add description of the average individual
DescriptCplot(k = kappa_hd_mean,
              ndata = ndata/2,
              refline = 0,
              denscol = NA,
              pcol = NA,
              sdcol = NA,
              mvcol = col_obs
)
Draws2Cont(draws = draws_treat,
          x_string = 'sin(b_beta0_Intercept)*
            A1(softplus(b_kappa_Intercept))',
          y_string = 'cos(b_beta0_Intercept)*
            A1(softplus(b_kappa_Intercept))',
          ngrid = 100, # needs higher resolution to avoid spillover
          cropc = TRUE
)
with(draws_treat,
     arrows.circular(x = mean.circular(circular(b_beta0_Intercept,
                                                units = 'radians',
                                                rotation = 'clock',
                                                zero = pi/2)
     )[1],
     y = A1(softplus(median(b_kappa_Intercept))),
     lwd = 2,
     length = 0.1/1.25,
     col = adjustcolor(col_sd, alpha.f = 200/255))
)


#estimate somewhat favours a turn
with(draws_treat,
     paste0(mean(b_beta0_x > 0)*100, '%')
)
```

```
## [1] "99.35%"
```

```
with(draws_treat,
     VertHist(data = unwrap_circular_deg(b_beta0_x),
              main = 'change in mean angle',
              ylim = c(-180, 180),
              col = adjustcolor(col_sd, alpha.f = 100/255),
              cex.axis = 0.7,
              axes = FALSE))
axis(side = 1)
axis(side = 2,
     at = -6:6*(180/6) )
abline(h = 0,
       col = 'gray',
       lwd = 7)
```



Summary

- Circular GLMs can be used to model effects on mean direction μ and concentration κ.
- The posterior density of parameter estimates can be compared with expected values; for example, an expected angle (such as the 'home direction') or concentration (such as $\kappa = 0$ for the circular uniform distribution).
- Where expected values of μ and κ are not known, fitted estimates can be compared between data for control and treatment conditions as an offset parameter between the two.
- Grouping factors, such as individual biases, can be accounted for in a hierarchical model, and the properties of the population of individuals estimated as hyperparameters for the concentration of

individual mean directions (κ_μ) and standard deviation of individual concentrations (σ_κ).

- Competing hypotheses about effects within a dataset can be formalised as circular GLMs with differing parameters, and their plausibility contrast directly *via* model-comparison methods such as expected log predictive density.